

SOFTWARE TESTING METHODOLOGIES

UNIT I

Software Testing: Introduction, Evolution, Myths & Facts, Goals, Psychology, definition, Model for testing, Effective Vs Exhaustive Software Testing.

Software Testing Terminology and Methodology: Software Testing Terminology, Software Testing Life Cycle, Software Testing Methodology.

Verification and Validation: Verification & Validation Activities, Verification, Verification of Requirements, High level and low level designs, verifying code, Validation.

SOFTWARE TESTING:

1. Introduction
2. Evolution
3. Myths & Facts
4. Goals
5. Psychology
6. Definition
7. Model for testing
8. Effective Vs Exhaustive Software Testing.

In this should be able to understand:

1. How software testing has evolved over the years
2. Myths and facts of software testing
3. Software testing is a separate discipline
4. Testing is a complete process
5. Goals of software testing
6. Testing is based on a negative/destructive view
7. Model for testing process
8. Complete testing is not possible
9. Various schools of software testing

1.1. SOFTWARE TESTING -INTRODUCTION

1. Software has become an essential component of any electronic device or system.
2. Job trends have shifted from development to software testing.
3. software testing is a complete process today, it must be measured to check whether the process is suitable for projects. The testing process should also be 'measured' using models & metrics to make sure of its quality and levels. CMM (Capability Maturity Model) has measured the development process on a scale of 1–5 and companies are running for the highest scale.
4. But many people in the industry/academia don't realize the need for measuring the testing, its level, and its place in the model being used. A TMM also exists that measures the maturing status of the testing process.
5. Testing is an important part of the software development life cycle (SDLC). Separate testing groups should come into existence, more in number, to divide the tasks among them so as to identify the errors in a pin-pointed manner and also suggest the remedies to get rid of them.
6. It should also be noted that testing and debugging can't be carried out 100%. It is better to come out with *effective testing* instead of *exhaustive testing*.

1.2. SOFTWARE TESTING -EVOLUTION

1. software testing was considered only a debugging process for removing errors after the development of software.
2. By 1970, the term 'software engineering' was in common use. But software testing was just a beginning at that time. In 1978, G. J. Myers realized the need to discuss the techniques of software testing in a separate subject.
4. By 1980, software professionals and organizations started emphasizing on quality. Organizations realized the importance of having quality assurance teams to take care of all testing activities for the project right from the beginning.
5. In the 1990s, testing tools finally came into their own. There was a flood of various tools, which are absolutely vital to adequate testing of software systems. However, they do not solve all problems and cannot replace a testing process.
6. Gelperin and Hetzel [79] have characterized the growth of software testing with time. Based on this, we can divide the evolution of software testing into the following phases.

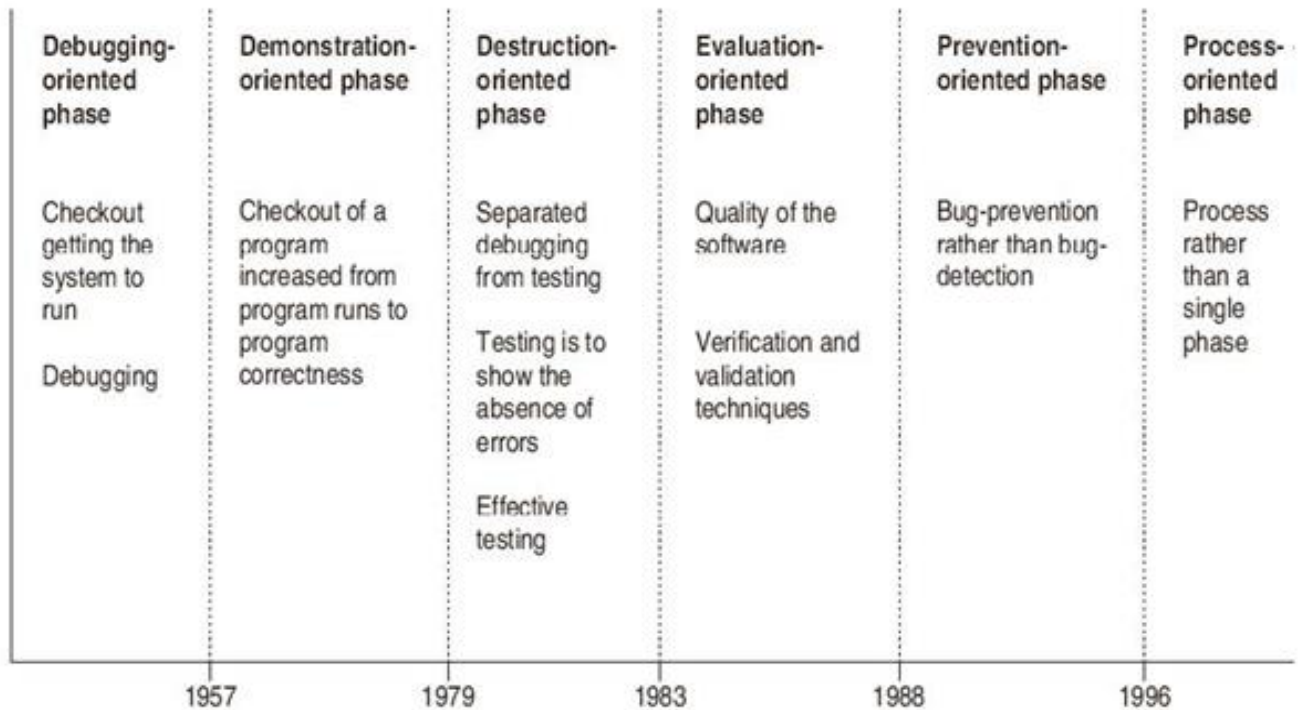


Figure 1.1 Evolution phases of software testing

1. Debugging Oriented Phase (Before 1957) :

1. In this early period of testing, Software Testing basics were unknown. Programs were written, run, tested and debugged by programmers until they were sure that all the bugs were removed.
2. The term 'checkout' was used for testing. The term used for testing was checkout, focused on getting the system to run.
3. Debugging was a more general term at that time and it was not distinguishable from software testing. Till 1956, there was no clear distinction between software development, testing, and debugging.

2. Demonstration-oriented Phase (1957–78)

1. The term 'debugging' continued in this phase. in 1957, Charles Baker pointed out that the purpose of checkout is not only to run the software but also to demonstrate the correctness according to the mentioned requirements.
2. The scope of checkout of a program increased from program runs to program correctness. Moreover, the purpose of checkout was to show the absence of errors.
3. There was no stress on the test case design.

3. Destruction-oriented Phase (1979–82)

1. This phase is a revolutionary turning point in the history of software testing.

2. Myers changed the view of testing from ‘testing is to show the absence of errors’ to ‘testing is to find more and more errors.’ separated debugging from testing and stressed on the valuable test cases if they explore more bugs.
3. This phase has given importance to effective testing in comparison to exhaustive testing.
4. The importance of early testing was also realized in this phase.

4.Evaluation-oriented Phase (1983–87)

1. This phase stresses on the quality of software products such that it can be evaluated at every stage of development.
2. In fact, the early testing concept was established in the form of verification and validation activities which help in producing better quality software.
3. In 1983, guidelines by the National Bureau of Standards were released to choose a set of verification and validation techniques and evaluate the software at each step of software development.

5.Prevention-oriented Phase (1988–95)

1. This phase more importance to the point that instead of detecting the bugs, it is better to prevent them by experience.
2. Bugs can also be prevented in other projects with the experience gained in similar software projects.
3. The prevention model includes test planning, test analysis, and test design activities playing a major role, while the evaluation model mainly relies on analysis and reviewing techniques other than testing.

6.Process-oriented Phase (1996 onwards)

1. In this phase, testing was established as a complete process rather than a single phase (performed after coding) in the software development life cycle (SDLC).
2. The testing process starts as soon as the requirements for a project are specified and it runs parallel to SDLC.
3. The model for measuring the performance of a testing process has also been developed like CMM. The model for measuring the testing process is known as Testing Maturity Model (TMM).
4. The evolution of software testing was also discussed by Hung Q. Nguyen and Rob Pirozzi three phases, namely Software Testing 1.0, Software Testing 2.0, and Software Testing 3.0. According to this classification, the current state-of-practice is Software Testing 3.0.

ANOTHER ANGLE OF SOFTWARE TESTING EVOLUTION STATES 3 PHASES:

Software Testing 1.0: In this phase, software testing was considered a single phase to be performed after coding of the software in SDLC. No test organization was there.

A few testing tools were present but their use was limited due to high cost.

Management was not concerned with testing, as there was no quality goal.

Software Testing 2.0

In this phase, software testing gained importance in SDLC and the concept of early testing also started. Testing was evolving in the direction of planning the test resources. Many testing tools were also available in this phase.

Software Testing 3.0

In this phase, software testing is being evolved in the form of a process which is based on strategic effort.

It was established that an overall process should be present to give a roadmap of all testing techniques and methodologies.

It should be driven by quality goals all controlling and monitored by the managers. Thus, the management is actively involved in this phase

1.3.SOFTWARE TESTING—MYTHS AND FACTS

Myth1:Testing is a single phase of SDLC.

Truth1: In reality, testing starts as soon as we obtain the requirements/specifications .Testing continues through out the SDLC and even after the implementation of the software.

Myth2:Testing is easy.

Truth2: Testing tools automate some of the tasks but not test cases' designing. A tester has to design the whole testing process and develop the test cases manually – which means that he has to understand the whole project and process. Typically, a tester's job is harder than that of a developer.

Myth3:Software development is more important than testing.

Truth3: This myth exists from the beginning of a career and testing is considered a secondary job. These days ,testing surfaced as a complete process ,same as that of development .A testing team enjoys equal status to that of a development team.

Myth4:Complete Testing is possible.

Truth4:Any person who has not worked on test cases might feel this. In reality, complete testing is never possible. All the inputs can't be perceived and provided since it makes the testing process too large. Hence, 'complete/exhaustive' testing has been replaced by 'effective' testing.

NOTE: Effective testing selects and runs some important test cases so that severe bugs are uncovered first.

Myth5:Testing starts after program development.

Truth5: This idea is not proper. A tester performs testing at the end of SDLC in of every phase the form of

verification and validation testing. Detailed test plans and cases are prepared and reports are forwarded to the developers' team. Testing after coding is just a small part of the testing process.

Myth6: The purpose of testing is to check the functionality of the software.

Truth6: The goal of testing is to ensure quality of the software. Quality doesn't mean checking the functionalities of all the modules; many other points are also to be taken in to account.

NOTE: Quality=>Standards; functionality=>different operations to be carried out

Myth7: Anyone can be a tester.

Truth7: Testing can be performed only after a tester is properly trained for various purposes like understanding the SDLC phases designing the test cases and learning about various testing tools. Companies give preference to certified testers.

1.4. GOALS OF SOFTWARE TESTING

The goals of software testing may be classified into three major categories, as shown in Fig. 1.2.

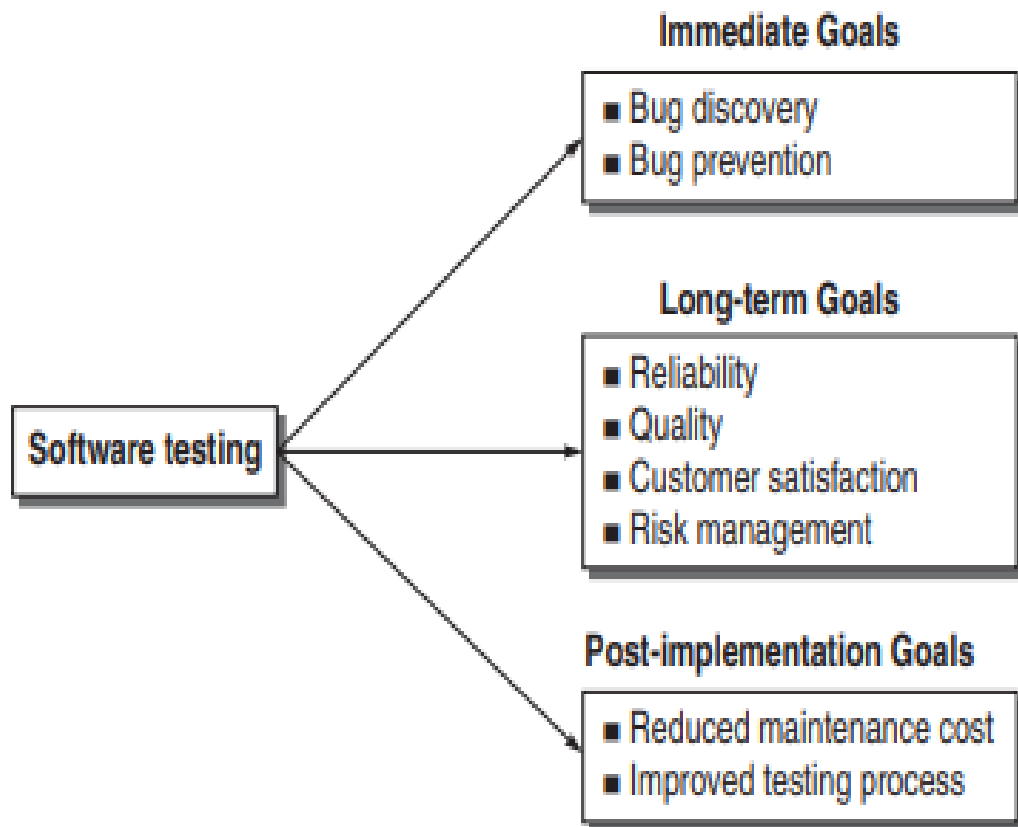


Figure 1.2 Software testing goals

The goals of ST can be classified in to three major categories:

1.Short Term or Immediate Goals: These goals are the immediate results after performing testing procedure. They can beset in the individual phases of SDLC.

Bug Discovery: Find errors at any stage of software development. More bugs discovered at an early stage better will be the success rate of software testing.

Bug Prevention: It is the consequent (result) action of bug discovery. From the discovery, behavior, and interpretation of the bugs discovered, everyone in the software development team gets to learn how to code safely such that the bugs discovered should not be repeated in later stages or future projects. errors cannot be prevented to zero, they can be minimized.

Bug prevention is a superior goal of testing.

2.Long Term Goals: These affect the ‘product quality’ in the long run, when a cycle of the SDLC is completed.

1.Quality: software is also a product, its quality is primary from the users’ point of view.

Through testing ensures superior quality of the software.

The first goal of understanding and performing the testing process is to enhance the quality of the software product.

Quality depends on various factors like technical correctness, integrity, efficiency, etc., reliability is the major factor to achieve quality.

2.Reliability: Reliability is a matter of confidence that the software will not fail – and confidence increases with rigorous testing, thus increasing reliability and quality.

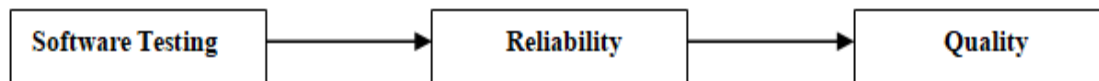


Fig : Testing produces reliability and quality

3.Customer Satisfaction:

Testing must satisfy the user for all specified requirements and other unspecified requirements also.

The customer to be satisfied with the software product, then testing should be complete and thorough.

Testing should be complete in the sense that it must satisfy the user for all the specified requirements mentioned in the user manual, as well as for the unspecified requirements which are otherwise understood.

A complete testing process achieves reliability, reliability enhances the quality, and quality in turn, increases the customer satisfaction.

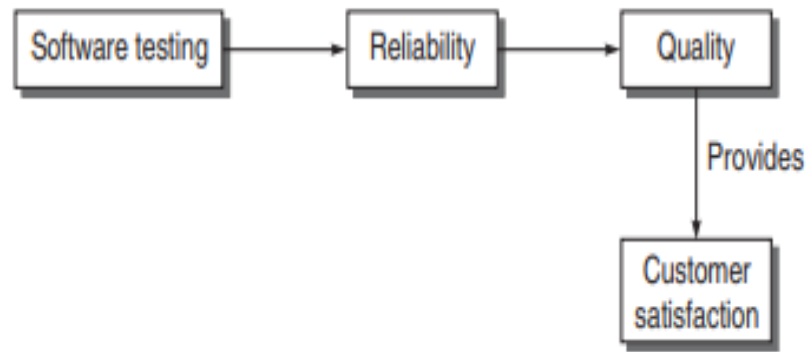


Figure 1.4 Quality leads to customer satisfaction

4.Risk Management:

Risk is the probability that undesired events will occur in a system leading to unsuccessful termination of goals or project.

Risk is basically concerned with the business perspective of an organization.

Risks must be controlled by Software Testing (as a control) which can help in minimizing or terminating risks. The purpose of ST as a control is to provide information to management so that they can react in a better way to risk situations. Ex: Time is less, more chance of finding bugs etc.

A tester's responsibility is to evaluate business risks and make them a basis for choosing the tests.

Testers should also categorize the Levels of the risks should be categorized as high, moderate, low so that it becomes a foundation for designing the test cases.

Risk management becomes a long term goal for ST and has to be dealt with care fully to extract positive results.

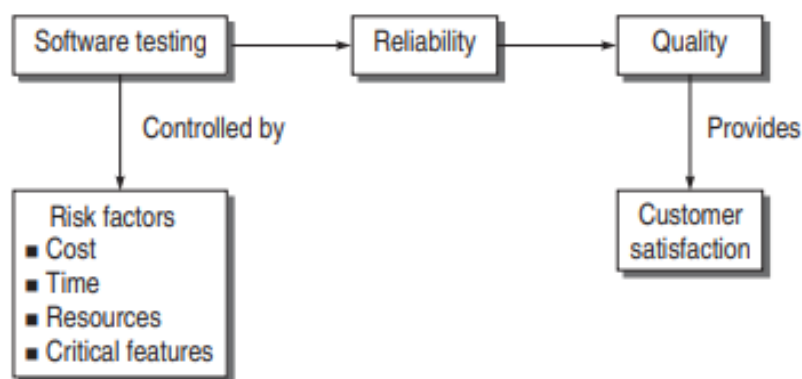


Figure 1.5 Testing controlled by risk factors

3.Post-implementation Goals:[After the product is released]

Reduced maintenance Cost: In software market, The maintenance cost of any software product is not its physical cost, as the software does not wear out.

The only maintenance cost in a software product is its failure due to errors. Post release errors are more costly since that are difficult to trace and fix.

If testing had been done rigorously, chances of failures are minimized ,thus reducing maintenance cost.

Improved software testing process:

A testing process for one project may not be successful and there may be scope for improvement.

Therefore, the bug history and post-implementation results can be analyzed to find out snags in the present testing process, which can be rectified in future projects.

The long-term post-implementation goal is to improve the testing process for future projects.

1.5.PSYCHOLOGY FOR SOFTWARE TESTING

Software Testing is directly related to human psychology.

“Testing is the process of demonstrating that there are no errors”. [positive approach]

The purpose of testing is to show that the software performs its intended functions correctly.

This definition is correct, but partially. If testing is performed keeping this goal in mind, then cannot achieve the desired goals first identified this approach of testing the software.

This approach is based on the human psychology that human beings tend to work according to the goals fixed in their minds. assumption that the software is error-free, then consequently, will design the test cases to show that all the modules run smoothly.

This ‘negative approach’ is taken up for the benefit of both developer and tester and the result is that many unexpected bugs might get found. This methodology is *effective testing*.

“Testing is the process of executing a program with the intent of finding errors”.

Thus Software Testing can be defined as a process of executing a program with the intent of finding maximum errors.

Testing can be compared with the analogy of medical diagnostics of a patient. If lab test sare negative, it can’t be regarded as a successful test. If they are positive, then the doctor can start appropriate treatment immediately.

1.6.SOFTWARE TESTING DEFINITIONS:

“Testing is the process of executing a program with the intent of finding errors.” --- Myers

“A successful test is one that uncovers an as-yet-undiscovered error.” --- Myers

“Testing can show the presence of bugs but never their absence”. ----W. Dijkstra

“Testing is a support function that helps developers look good by finding their mistakes before anyone else does”. ---- James Bach

“ST is an investigation conducted to provide information about the quality of the product WRT the operating context”.

“ST is a concurrent life cycle process of engineering, using, and maintenance of testing methodologies in order to improve the quality of the concerned software being tested”.

Thus, finally software testing can be defined as,

“Software testing is a process that detects important bugs with the objective of having better quality software.”

NOTE: Quality is the primary goal of testing; testing team is not responsible for QA. Finally ,ST can be defined as a process that detects important bugs with the objective of having better quality software.

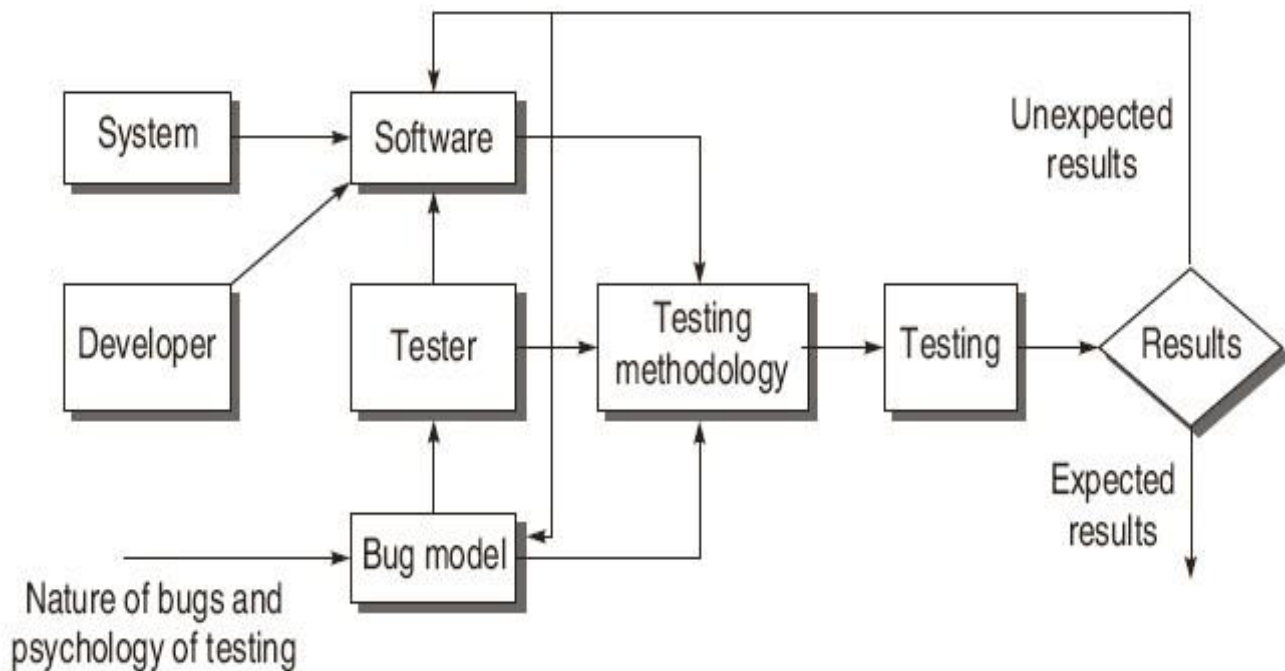
1.7.MODEL FOR SOFTWARE TESTING:

The software is basically a part of a system for which it is being developed.

The developer develops the software in the prescribed system environment considering the testability of the software.

Testability is a major issue for the developer while developing the software, as a badly written software may be difficult to test.

Fig: Software testing model



Testers are supposed to get on with their tasks as soon as the requirements are specified.

With suitable testing techniques decided in the testing methodology, testing is performed on the software with a particular goal.

The following describe the testing model:

1. Software and Software Model:

Software is built after analyzing the system in the environment.

It is complex entity which deals with environment, logic, programmer psychology etc.

Since in this model of testing, our aim is to concentrate on the testing on the testing process, therefore the software under consideration should not be complex such that it would not be tested.

Thus, the software to be tested may be modeled such that it is testable, avoiding unnecessary complexities.

2. Bug Model:

- Bug model provides apperception of the kind of bugs expected, considering the nature of all types of bugs, a bug model can be prepared that may help in deciding a testing strategy.
- However, every type of bug cannot be predicted. Therefore, if we get incorrect results, the bug model needs to be modified.

3. Testing Methodology and Testing:

1. Based on the inputs from the software model and the bug model, testers can develop a testing methodology that incorporates both testing strategy and tactics.
2. Testing strategy is the roadmap that gives us well-defined steps for the overall testing process. Once the planned steps of the testing process are prepared, software testing techniques and testing tools can be applied within these steps.
3. Once the planned steps of the testing process are prepared, software testing techniques and testing tools can be applied within these steps. Thus, testing is performed on this methodology.
4. Test plan/methodology is based on both the s/w (SDLC) and bug models. It gives well-defined steps for the overall testing process, taking care of the risk factors also.
5. Once a testing methodology is in place, test cases can be designed and testing tools can be applied at various steps. If we don't obtain the expected results, modifications are apparently necessary.

1.8. EFFECTIVE SOFTWARE TESTING VS. EXHAUSTIVE SOFTWARE TESTING

1. Exhaustive or complete software testing means that every statement in the program and every possible path combination with every possible combination of data must be executed. But will realize that exhaustive testing is out of scope.
2. The testing process should be understood as a domain of possible tests. There are subsets of these possible tests.
3. But the domain of possible test becomes infinite, cannot test every possible combination.

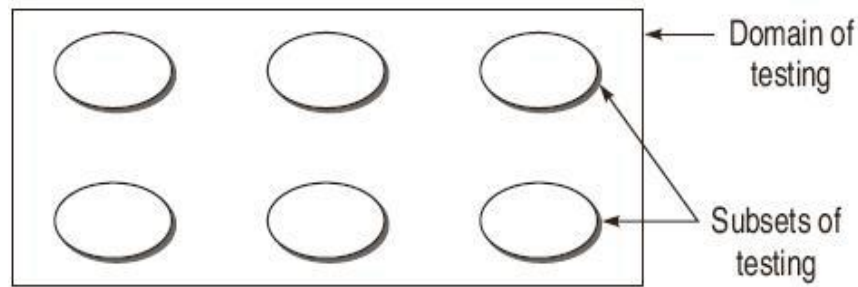


Fig :Testing domain

Complete testing requires the organization to invest along time which is not cost-effective. Therefore, testing must be performed on selected subsets ,that can be performed with in the constrained resources. This selected group of subsets, but not the whole domain of testing, makes effective testing.

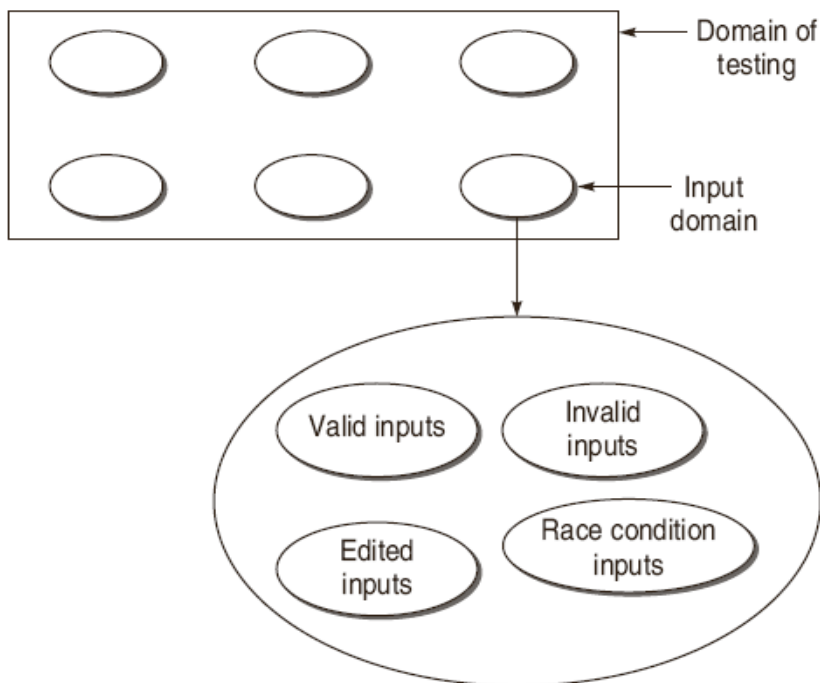
Available computer speed and the time constraints limit the possibility of performing all the tests. Hence, testing must be performed on selected subsets with constrained resources.

Effective testing can be enhanced if subsets are selected based on the features that are important for the concerned s/w environment.

Now let us see in detail why complete testing is not possible:

The domain of possible inputs to the Software is too large to test:

Fig: Input Domian for Testing



consider the input data as the only part of the domain of testing, even then are not able to test the complete input data combination.

The domain of input data has four sub-parts:

1.Valid Inputs: It seems that we can test every valid input on the software.

But look at a very simple example of adding two-digit two numbers. Their range is from -99 to 99 (total 199). So the total number of test cases combinations will be $199 \times 199 = 39601$.

2.Invalid Inputs: The important thing in this case is the behaviour of the program as to how it responds when a user feeds invalid inputs. If we consider the example of adding two numbers, then the following possibilities may occur from in valid inputs:

- Numbers out of range,
- Combination of Alphabets and digits,
- Combination of all Alphabets,
- Combination of Control characters,
- Combination of any other key on the keyboard.

3.Edited Inputs: If we can edit inputs at the time of providing inputs to the program, then many unexpected input events may occur. For Example, you can add many spaces to the input, which are not visible to the user.

4.Race Condition Inputs: The timing variation between two or more inputs is also one of the issues that limit the testing .For example, there are two input events A and B. According to the design A precedes B in most cases, but B can also come

First in rare and restricted condition This is the race condition whenever B precedes A. Race conditions are among the least tested.

There are too many paths through the Program to Test:

1. A program path can be traced through the code from the start of a program to its termination.
2. A testing person thinks that if all the possible paths of the control flow through the program are executed, then possibly the program can be said completely tested.
3. A testing person thinks that if all the possible paths of control flow through the program are executed, then the program is said to be completely tested.

There are two flaws in the statement.

i) The number of unique logic paths through a program is too large .n example shown in Fig. 1.9. It depicts a 10–20 statements program consisting of a DO loop that iterates up to 20 times. Within the body of the DO loop is a set of nested IF statements. The number of all the paths from point A to B is approximately 10^{14} . Thus, all these paths cannot be tested, as it may take years of time.

ii) The complete path testing, if performed somehow does not guarantee that there will be no errors.

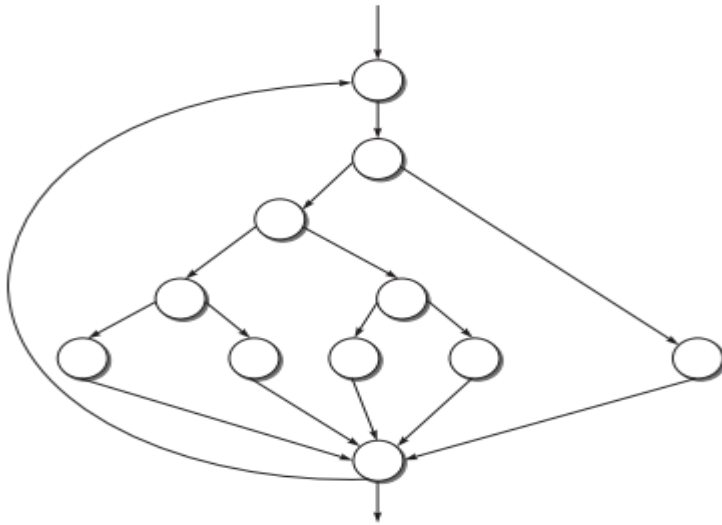


Figure 1.9 Sample flow graph 1

However there are two flaws in the statement:

Consider the following segment : `for(i=0;i<n: i++)`

```

{
if(m>=0)
x[i] =x[i] +10;
else
x[i] =x[i]- 2;
}
  
```

Fig :sample code fragment

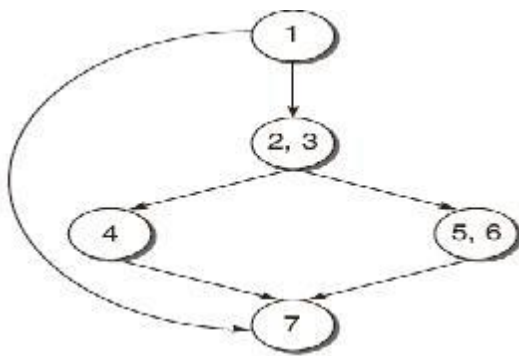


Fig: Example flow graph

- if n is 20, then the number of paths will be $220 + 1$, i.e. 1048577.

In our example, there are two paths in one iteration. Now the total number of paths will be $2^n + 1$, where n is the number of times the loop will be carried out, and 1 is added for -for loop to terminate. Thus if n is 20, the number of paths will be 1048577.

The complete path testing, if performed somehow, does not guarantee that there will not be errors. For example, if a programmer develops a descending order program in place of ascending order program then exhaustive path testing is of no use.

Exhaustive testing	Effective testing
Exhaustive testing aims at complete testing i.e no scope of further bug detection should be possible.	Effective testing is a smart way of prioritizing and covering all critical test cases that too with resource constraints.
It is impractical due to various reasons like time constraints, inadequate resources, achieving deadlines, etc.	It is a practical method as all deadlines can be met with almost no defects or issues in the final product.
It is not cost-effective.	It is a cost-effective method.
The whole process is quite complex and can be achieved for only smaller projects.	The process is not that complex nor time-consuming.

II Software Testing Terminology and Methodology:

1. Software Testing Terminology
2. Software Testing Life Cycle
3. Software Testing Methodology

1. Software Testing Terminology

1. Definitions
2. Life Cycle of a Bug
3. States of a Bug
4. Why do Bugs Occur
5. Bugs Affect Economics of Software Testing
6. Bug Classification based on Criticality
7. Bug Classification based on SDLC
8. Testing Principles

Software Testing Terminology Terms like error ,bug ,failure ,defect etc. are not synonymous and should be used carefully as the situation demands.

2.1.1. Definitions

Failure: It is the inability of a system/component to perform a required function according to its specification. It means results of a test are different from the expected ones. Failure depicts problems in a system on the output side.

Fault/Defect/Bug: Fault is a condition that causes the system to produce a failure. Fault is synonymous with the words defect and bug.

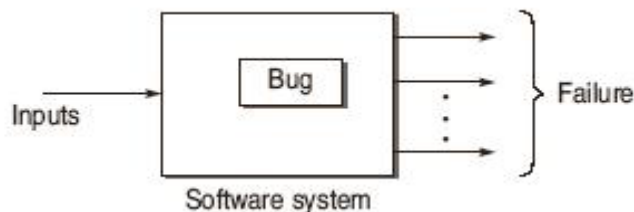


Fig: Testing Terminology

Fault is the reason embedded in any phase of SDLC and results in a failure.

It should also be noted that hidden bugs(unexecuted ones) may also lead to failures in future.

Error: If mistakes are made at any phase of the SDLC, errors are produced .It is a term generally used for human mistakes. An error causes a bug and bug in turn causes failures.



Fig:Flow of faults

“A mistake in coding is called error, error found by tester is called defect, defect accepted by development team then it is called bug ,build does not meet the requirements then it Is failure.”

Example: Consider the following module in a software:

Module A ()

```

{
.....
While(a>n +1);
{
....
Printf ("The value of x is:", x);
}
....
}
  
```

Expected Output: Value of x is printed.

Obtained Output: Value of x is not printed; may get warning/error.

This is a failure of the program.

Reason : The ‘while’ loop is not executed due to the presence of a semi-colon at the end of the statement. This is an error.

Example:

```

#include< stdio .h>int main ()
{
int value1, value2, ans;value1=5;
value2=3;
ans =value1-value2;
printf("The addition of 5 + 3 = %d.", ans);return0;
}
  
```

}

When you compile and run this program , the printed statement as below:

The addition of 5 +3 =2.

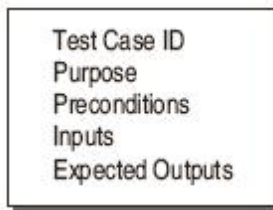
So after compiling and running this program realize the program has failed to do what it was supposed to do. The program was supposed to add two numbers but it certainly did not add 5 and 3.5+3 should be 8, but the result is 2. There could be various reasons as to why the program displays the answer 2 instead of 8. For now we have detected a *failure*. As the failure has been detected a *defect* can be raised.

Test Case:

Test Case is well documented procedure designed to test the functionality of a feature in the system.

A primary purpose of designing test case is to find errors in the system.

For designing the test case it needs to provide a set of inputs and its corresponding expected outputs.



Test case ID is the identification number given to each test case.

Purpose defines why the case is being designed.

Preconditions for running the inputs in a system can be defined.

Inputs should not be hypothetical. Actual inputs must be provided instead of general inputs.

Expected outputs are the outputs which should be produced when there is no failure.

(a)Test ware: The documents created during testing activities are known as test ware .These are the documents created during testing activities and those that a test engineer produces .They include test plans, test specifications ,test case design ,test reports etc. All the Test ware should be managed and updated like a software product.

(b)Incident: Incident is a symptom that indicates a failure has taken place or is going to take place.It alerts a user about the future coming failure.

(c) Test oracle: An oracle is the means to judge the success or failure of a test i.e. to judge the correctness of the system for some test. The simplest oracle is comparing actual results with expected results by hand.

Example :Let us say that we need to check an input field that can accept maximum of 10 characters. While developing the test cases for the above scenario, the test cases are documented the following way. In the below example ,the first case is a pass scenario while the second case is a FAIL.

Scenario	Test Step	Expected Result	Actual Outcome
Verify that the input field that can accept maximum of 10 characters	Login to application and key in 10 characters	Application should be able to accept all 10 characters.	Application accepts all 10 characters
Verify that the input field that can accept maximum of 11 characters	Login to application and key in 11 characters	Application should NOT accept all 11 characters.	Application accepts all 10 characters.

If the expected result doesn't match with the actual result, then we log a defect. The defect goes through the defect lifecycle and the testers address the same after fix.

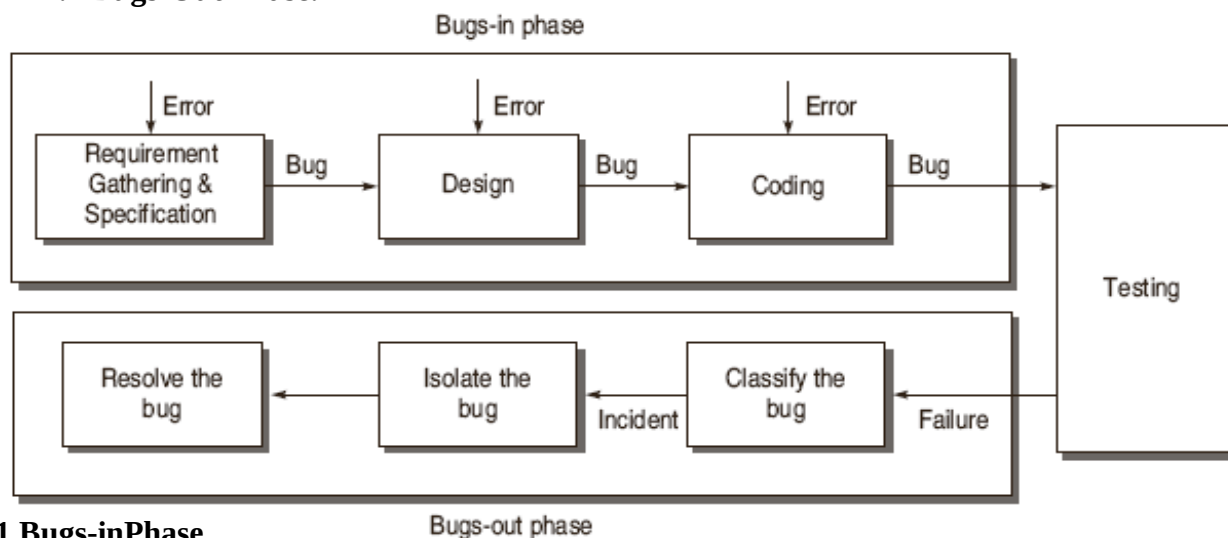
2.1.2. Life Cycle of a Bug

Developer can make an error in any phase of SDLC. If an error has been produced in one phase and not detected in the same phase, then it results as a bug in next phase.

The whole life cycle of a bug can be classified into two phases:

1. Bugs-In Phase And

2. Bugs-Out Phase.



1. Bugs-inPhase

- This phase is where the errors and bugs are introduced in the software.
- This bug is creates errors on a specific location of the software and consequently, when this error goes unnoticed, it causes some conditions to fail, leading to a bug in the software.
- This bug is carried out to the subsequent phases of SDLC, if not detected.
- Thus a phase may have its own errors as well as bugs received from the previous phase.

2.Bugs-Out Phase:

- If failures occur while testing a software product, we come to the conclusion that it is affected by bugs.
- When we observe failures,

The following activities are performed to get rid of the bugs:

1.Bug Classification:

- In this part, we observe the failure and classify the bugs according to its nature.
- A bug can be critical or it may have no adverse effect on the behavior of the software.
- This classification may help the developer to prioritize the handling of the bugs.

2.Bug Isolation:

- Bug isolation is the activity by which we locate the module in which the bug appears.
- Incidents observed in failures help in activity.
- Through the symptoms observed, and back-trace the design of the software and reach the file/module and the condition inside it which has caused the bug; this is known as the isolation of the bug.

3.Bug Resolution:

- After the bug is isolated, back-trace (reverse engineering) the design to pinpoint the location of the error.
- A bug is resolved when we have found the exact location of its occurrence.

2.1.3.STATES OF A BUG

Bug attains following different states in its life cycle.

States of a Bug:

New :The state is new when the bug is reported first time by a tester.

Open : After a tester has posted a bug, the test leader approves that the bug is genuine and he changes the state as OPEN.

Assign: Once the leader changes the state as OPEN, he assigns the bug to corresponding developer or development team and state of bug is changed to ASSIGN.

Test:

- Once the developer fixes the bug, to assign the bug to the testing team for next round of testing.
- Before releases the software with bug fixed, he changes the state to TEST. It means that bug has been fixed and is released to testing team.

Deferred:

- The developer who has been assigned to fix the bug will check its validity and priority.

- If the priority of the reported bug is not high or there is not sufficient time to test it or the bug does not have any adverse effect on the software, then the bug is changed to deferred state which implies the bug is expected to be fixed in next releases.

Rejected: If the developer feels that the bug is not genuine, he rejects the bug. Then the state of the bug is changed to REJECTED.

Duplicate: If the bug is repeated twice or the two bugs mention the same concept of the bug, then one bug status is changed to DUPLICATE.

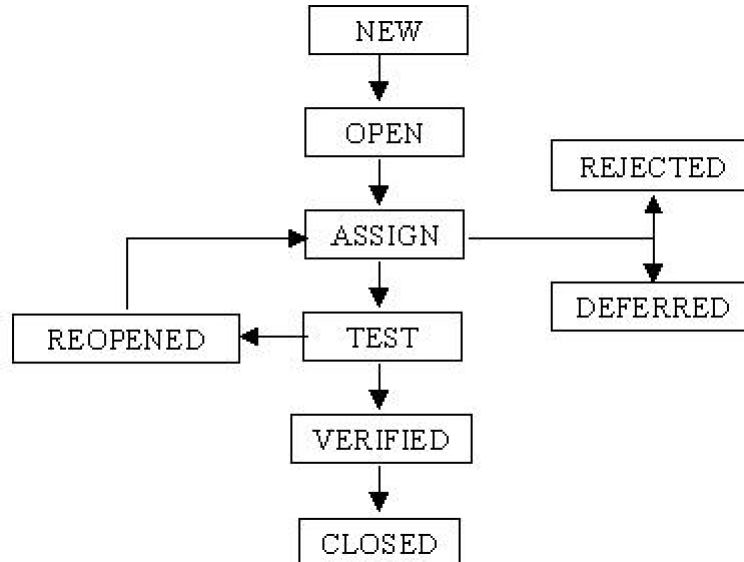
Verified: Once the bug is fixed and the status is changed to TEST, the tester tests the bug. If the bug is not present in the software, he approves that the bug is fixed and changes the status to VERIFIED.

Reopened : If the bug still exists even after the bug is fixed by the developer, the tester changes the status to REOPENED. The bug traverses the life cycle once again.

Closed:

- Once the bug is fixed, it is tested by the tester.
- If the tester feels that the bug no longer exists in the software, changes the status of the bug to CLOSED.
- This state means that the bug is fixed, tested and approved

Figure 2.5 States of a bug



2.1.4. Why do Bugs Occur:

- A software bug may be defined as a coding error that causes an unexpected defect, fault, flaw, or imperfection in a computer program.
- In other words, if a program does not perform as intended, it is most likely a bug.
- There are bugs in software due to unclear or constantly changing requirements, software complexity, programming errors, timelines, errors in bug tracking, communication gap, documentation errors, deviation from standards etc.

1.To Err is Human.

- Errors are produced when developers commit mistakes.
- For example, the phone rang while coding and the developer got distracted, he pressed a wrong key and the results of that mistake produced a bug in the system.

2.Bugs in Earlier Stages go Undetected and Propagate

- The phases of SDLC are connected to each other and the output of one phase becomes the input to the next. Therefore, an undetected bug easily propagates into subsequent phases.
- These propagated bugs, in later phases, are harder to detect and if found, are costlier to debug. So testing should be performed at each level of SDLC.
- There are various reasons how do bugs get into a software product some examples are given below:
 1. Miscommunication in gathering requirements from the customer is a big source of bugs.
 2. Constantly changing software requirements cause a lot of confusion and pressure both on the development and testing teams. Often, a new feature added or existing feature removed can be linked to the other modules or components in the software.
 3. If the effect of changes in one module to another module is overlooked, it causes bugs.
 4. Rescheduling of resources, re-doing or discarding already completed work; changes in hardware/software requirements can affect the software too.
 5. Assigning a new developer to the project in midway can cause bugs. This is possible if proper coding standards have not been followed, improper code documentation, ineffective knowledge transfer etc.
 6. Complexity in keeping track of all the bugs can again cause bugs by itself. This gets harder when a bug has a very complex life cycle i.e. when the number of times it has been closed, reopened, not accepted, ignored etc goes on increasing.

2.1.5.Bugs Affect Economics of Software Testing:

- Testing prior to coding is 50% effective in detecting errors and after coding, it is 80% effective.
- It is at least 10 times as costly to correct an error after coding as before and 100 times as costly to correct a post release error. This is how the bugs affect the economics of testing.
- There is no guarantee that all the errors will be recognized after testing and that the testing phase will remove all of them. This is not possible practically. If the bugs embedded in earlier stages go undetected, it is more difficult to detect them in later stages .The cost to remove bugs from a system increases, as the cost of a bug is equal to Detection Cost + Correction Cost.
- Figure 2.6 outlines the relative cost to correct a defect depending on the SDLC in which it is discovered. Therefore, cost increase is logarithmic.

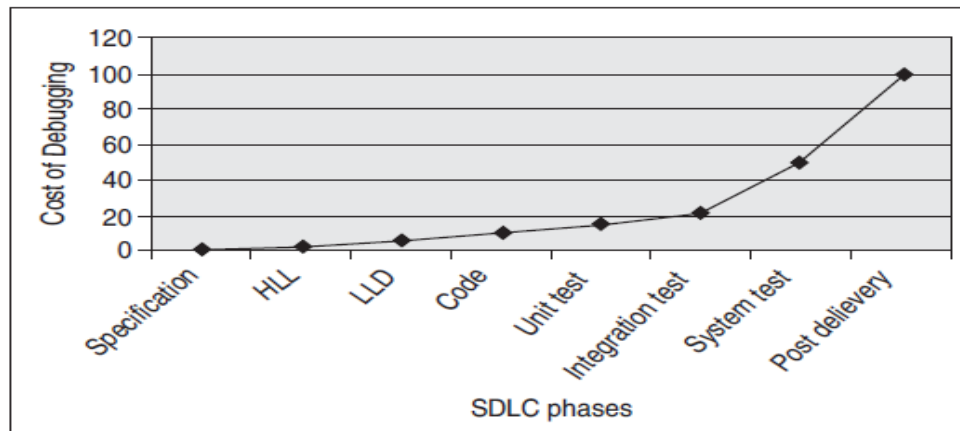


Figure 2.6 Cost of debugging increases if bug propagates

2.1.6.BUG CLASSIFICATION BASED ON CRITICALITY

- Bugs can be classified based on the impact they have on the software under test.
- This classification can be used for the prioritization of bugs.
- The assignment of Priority Levels during the product test phase includes:

1. Critical Bugs

- This type of bugs has the worst effect such that it stops or hangs the normal functioning of the software.
- The person using the software becomes helpless when this type of bug appears.
- For example, in a sorting program, after providing the input numbers, the system hangs and needs to be reset.

2. Major / High

- This type of bug does not stop the functioning of the software but it causes a functionality to fail to meet its requirements as expected.
- For example, in a sorting program, the output is being displayed but not the correct one.

3. Average / Medium

- Medium bugs are less critical in nature as compared to critical and major bugs.
- If the outputs are not according to the standards or conventions (something usually done), e.g. redundant or truncated output, then the bug is a medium bug.

4. Minor/Low

- These types of bugs do not affect the functionality of the software.
- These are just mild bugs which occur without any effect on the expected behavior or continuity of the software.
- For example, typographical error or misaligned printout.

2.7.BUG CLASSIFICATION BASED ON SDLC

Bugs can appear in any phase of SDLC. They are classified based on SDLC phases as following:

1.Requirements and Specification Bugs:

- The first type of bug in SDLC is in the requirement gathering and specification phase, most of the bugs appear in this phase only.
- If these bugs go undetected, they propagate in to subsequent phases.
- Requirement gathering and specification is a difficult phase because requirements gathered from the customer are to be converted into a **requirement specification** which will become the base for design.
- If specified requirements may be incomplete, ambiguous, or inconsistent. Specification problems lead to wrong missing, or superfluous features.

2. Design Bugs:

- Design bugs may be the bugs from the previous phase and in addition those errors which are introduced in the current phase. The following design errors may be there:

1.Control flow bugs:

- Control and sequence bugs include paths left out, unreachable code, improper nesting of loops, loop-back or loop termination criteria incorrect, missing process steps, duplicated processing, unnecessary processing, switches.

2.Logic bugs:

- Any type of logical mistakes made in the design is a logical bug.
- For example, improper layout of cases, missing cases, improper combination of cases, misunderstanding of the semantics of the order in which a Boolean expression is evaluated.

3.Processing Bugs:

- Any type of computation mistakes result in processing bugs.
- Examples include arithmetic error, incorrect conversion from one data representation to another, ignoring overflow, improper use of logical operators, etc.

4.Data-Flow Bugs:

- Control flow cannot identify data errors. For this, we use data flow (through data flow graph).
- There may be data flow anomaly errors like un-initialized data, initialized in wrong format, data initialized but not used, data used but not initialized, redefined without any intermediate use, etc.

5.ErrorHandlingBugs:

- If the system fails, then there must be an error message or the system should handle the error in an appropriate way.

Ex: Mishandling of errors ,no error messages etc.

6. Race Condition bugs: Race conditions also lead to bugs. Sometimes these bugs are irreproducible (not reproducible).

Ex: Every time ,the value of A is provided before B. Suddenly ,if we give B before A, the software system crashes.

7.Boundary related bugs: When the software fails at the boundary values, then these are known as boundary related bugs.

For example , there is one integer whose range is in between 1to 100.Usermusttestthe program by entering 0 or101.

8. User Interface bugs: If the user does not feel good while using the software, then there are user interface bugs. Ex: inappropriate error messages, missing info, misleading info etc.

Examples of UI bugs include inappropriate functionality of some feature, not doing what the user expects, wrong content in the help text etc....

9. Coding Bugs: Programmer is generally aware about coding bugs.

There may be a long list of coding bugs.

For example, undeclared data, undeclared routines, dangling code, typographical errors, documentation bugs, i.e. erroneous comments lead to bugs in maintenance.

3. Interface and Integration Bugs:

Various categories of bugs in Interface, Integration are:

1.External Interfaces: The external interfaces are the means used to communicate with the world .These include devices, actuators, sensors ,input terminals ,printers, and communication lines.

External interface bugs are : invalid timing or sequence assumptions related to external signals
Misunderstanding external input or output formats. In sufficient tolerance to bad input data.

2.Internal Interfaces: Integration bugs result from inconsistencies or incompatibilities between modules, data transfer bugs, data sharing bugs etc.

Internal interfaces are in principle not different from external interfaces but they are more controlled . A best example for internal interfaces are communicating routines.

4. System Bugs:

- System bugs cover all kinds of bugs that cannot be described to a component, interactions between any components such as programs, data, hardware, and the operating systems.
- System is based on various parameters like performance, stress, compatibility, usability, etc.
- For example, in a real-time system, stress testing is very important, as the system must work under maximum load.

- If the system is put under maximum load at every factor like maximum number of users, maximum memory limit, etc. and if it fails, then there are system bugs.

5. Testing Bugs:

- Testing is dedicated to finding bugs but the presence of bugs in the testing phase is also possible. But the fact is that bugs are present in testing phase also.
- Testing is also performed by testers – humans.
- Some testing mistakes are: failure to notice/report a problem, failure to use the test case, failure to make it clear how to reproduce the problem, failure to check for unresolved problems just before the release, failure to verify fixes, failure to provide summary report.

2.8 Testing Principles:

These principles are guidelines for a tester.

1. Effective Testing not Exhaustive Testing:

All possible combinations of tests become so large that is impractical to test them all (Exhaustive testing is not possible).

Therefore, the tester's approach should be based on effective testing to adequately cover program logic and all conditions in the component level design.

2. Testing is not a single phase performed in SDLC:

Testing is not just an activity performed after coding in SDLC,

Testing process starts as soon as specifications for the system are prepared and it continues till the release of product.

3. Destructive approach (irreparable) for constructive testing:

- Testers must have the psychology that bugs are always present in the program and they must think about the technique of how to uncover them but bugs can be negative/destructive approach.
- It has been proved that such a destructive approach helps in performing constructive and effective testing.
- A successful testing is to discover more and more bugs, and not to show that the system does not contain any bugs.

4. Early Testing is the best policy:

- Testing process is not a phase after coding; rather it starts as soon as requirement specifications are prepared.
- The cost of bugs can be reduced tenfold, as bugs are harder to detect in later stages if they go undetected. Thus, the policy in testing is to start as early as possible.

5. Probability of existence of an error in a section of a program is proportional to the number of errors already found in that section.

- Suppose that we found 50 errors in Module X, 12 in Module Y, and 3 in Module Z. The software was debugged but after a span of time, we find some errors again and the software is given to a tester for testing.
- This principle says that the tester should start with Module X which has the history of maximum errors.
- If some sections are found error-prone in testing, then our next testing effort should give priority to these error-prone sections.

6. strategy should start at the smallest module level and expand towards the whole program.

- Testing must begin at the unit or module level, gradually progressing towards integrated modules and finally the whole system.
- Testing cannot be performed directly on the whole system. It must start with the individual modules and slowly integrate the modules and test them. After this, the whole system should be tested.

Testing strategy should start at the smallest module level and expand toward the whole program.

Testing must begin at the unit or module level, gradually progressing towards integrated modules and finally whole system.

8. Testing should be performed by an independent team.

- Programmers develop the software; they test it at their individual modules. However, these Programmers are not good testers of their own software.
- They are basically constructors of the software, but testing needs a destructive approach.
- Therefore, it is always recommended to have the software tested by an independent testing team.
- Testers associated with the same project can also help in this direction, but this is not effective. For effective testing, the software may also be sent outside the organization for testing.

9. Everything must be recorded in software testing.

- Testing is not an intuitive (easy to use and understand) process; rather it is a planned process. It demands that every detail be recorded and documented.
- We must have the record of every test case run and the bugs reported. Even the inputs provided during testing and the corresponding outputs are to be recorded.
- Executing the test cases in a recorded and documented way can greatly help while observing the bugs.
- The experience with the test cases in one project can be helpful in other projects.

10.Invalid inputs and unexpected behavior have a high probability of finding an error.

- Whenever the software is tested, we test for valid inputs and for the functionality that the software is supposed to do.
- But we must test the software with invalid inputs and the behavior which is not expected in general. This is also a part of effective testing.

11.Testers must participate in specification and design reviews.

- Testers' role is not only to get the software and documents and test them.
- If they are not participating in other reviews like specification and design, it may be possible that either some specifications are not tested or some test cases are built for no specifications.

Example:

Let 'S' be the set of specified behaviors of the program, 'P' be the implementation of the program, and 'T' be the set of test cases.

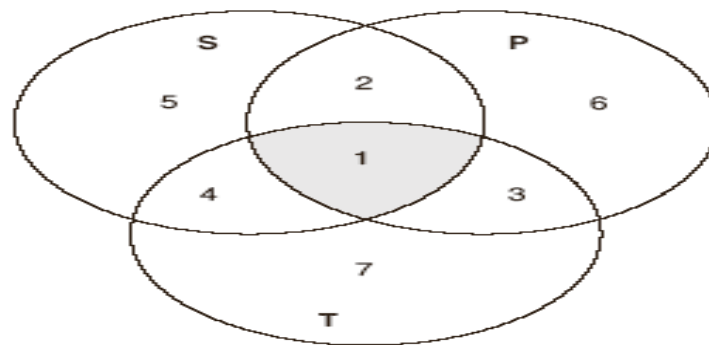


FIG: Venn diagram

There may be specified behaviours are not tested (regions 2 and 5).

Test cases that correspond to unspecified behaviours (regions 3 and 7).

Program behaviours that are not tested (regions 2 and 6).

S is the set of specified behaviours of the program

P is the implementation of the program

T is the set of test cases.

The good view of testing is to enlarge area of region 1. S, P and T must overlap each other such that all specifications are implemented and all of these are tested. This is possible only when the test team members participate in all discussions regarding specifications and design.

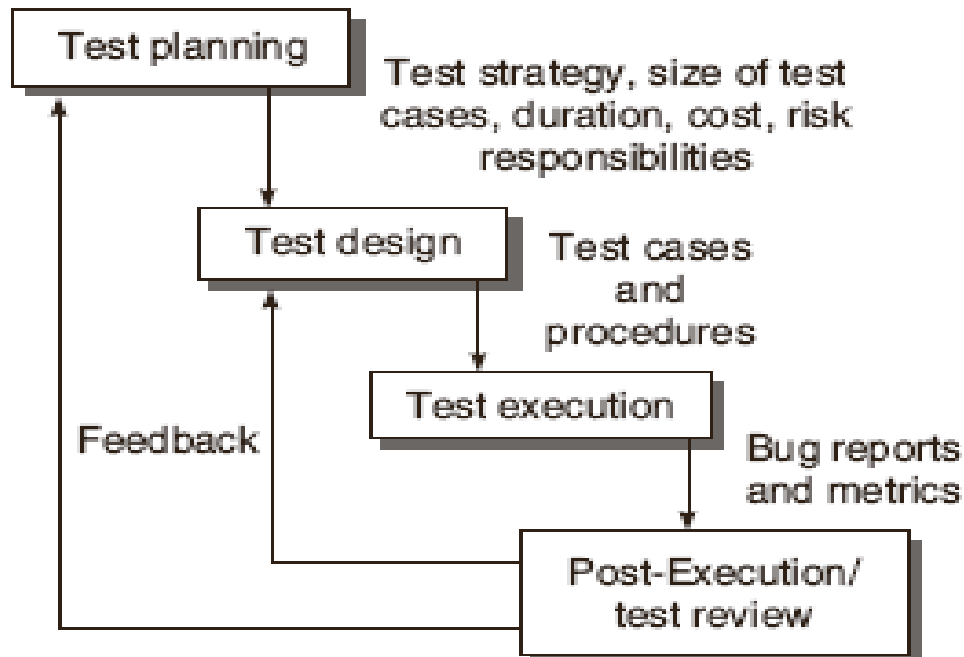
2.SOFTWARE TESTING LIFE CYCLE(STLC)

The testing process divided into a well-defined sequence of steps in termed as software testing life cycle(STLC).

The major contribution of STLC is to involve the testers at early stages of development.

This methodology involves the testers at early stages of development ,gives financial benefits and helps to reach significant milestones.

Fig: Software Testing life cycle(STLC)



1.Test Planning:

The goal of test planning is to take into account the important issues of testing strategy like resources, schedules, responsibilities, risks, priorities as roadmap.

The following are activities during test planning:

1. Defining the Test Strategy
2. Estimate of the number of test cases, their duration and cost.
3. Plan the resources like the manpower to test, tools required, documents required.
4. Identifying areas of risks.
5. Defining the test completion criteria.
6. Identification of methodologies, techniques and tools for various test cases.
7. Identifying reporting procedures, bug classification, databases for testing, Bug Severity levels, project metrics
 - Based on the planning, analysis is done for various testing activities.

- The major output of test planning is the *test plan document*. After analyzing the issues, the following activities are performed:
 1. Develop a test case formats.
 2. Develop test cases at different phases of SDLC.
 3. Identify test cases to be automated(if applicable)
 4. Prioritize the test cases according to their importance and criticality.
 5. Define areas of stress and performance testing.
 6. Plan the life cycles required for regression testing.

2.Test Design:

Design of test cases is the major activities in testing. It includes the following critical activities.

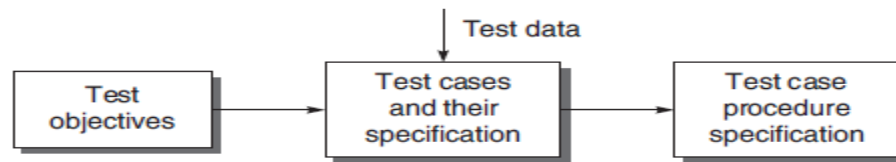


Fig: Test case design steps

1..Determining the test objectives and their Prioritization:

- The test objectives reflect the fundamental elements that need to be tested to satisfy an objective.
- For this purpose, you need to gather reference materials like SRS and design documentation.
- Then on the basis of reference materials a team of experts compile a list of test objectives.
- This list should also be prioritized depending upon the scope and risk.

2.Preparing List of Items to be Tested:

Objectives obtained are now converted into list of items that are to be tested under an objective.

3.Mapping items to test cases:

Test cases are identified for this list of items. For this purpose matrix is created to identify which test case will be covered by which item.

After making a list of test items to be tested, there is a need to identify the test cases.

A matrix can be created for this purpose ,identifying which test case will be covered by which item. The existing test cases can also be used for this mapping.

This matrix will help in:

Identifying the major test scenarios.

Identifying and reducing the redundant test cases.

Identifying the absence of a test case for a particular objective and as a result ,creating them.

Some attributes of a ‘good’ test case are given below:

It is designed considering the importance of test risks, thus resulting in good prioritization.

It has high probability of finding an error.

It reduces redundancy, overlapping and wastage of time.

It covers all features of testing of the concerned software, but in a modular approach.

A ‘successful’ test case is one that has discovered a completely new and unknown error.

4.Selection of Test case design techniques:

There are two broad categories namely black-box testing and white-box testing. The techniques to design test cases are selected such that there is more coverage and the system detects more bugs.

5. Creating Test Cases and Test Data:

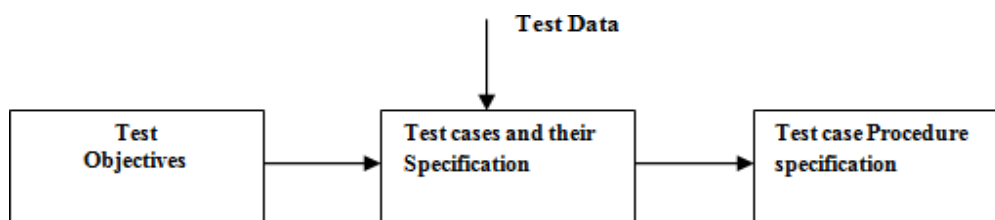
- The next step is to create test cases based on the testing objectives identified.
- The test cases mention the objective under which a test case is being designed, the inputs required, and the expected outputs.
- While giving input specifications, test data must also be chosen.

6.Setting up the test environment and supporting tools:

- The test created above needs some environment settings and tools, like hardware configurations, testers, interfaces, operating systems, and manuals must be specified during this phase.

7.Creating Test Procedure Specification : This is a description of how test case will be run. It is a form of sequenced steps. This procedure is actually used by the tester at the time of execution of test cases.

Fig: Test Case Design Steps



3.Test Execution:

In this phase, all test cases are executed.

- In this phase, all the test cases are executed including verification and validation.
- Verification test cases are started at the end of each phase of SDLC.
- Validation test cases are started after the completion of a module.

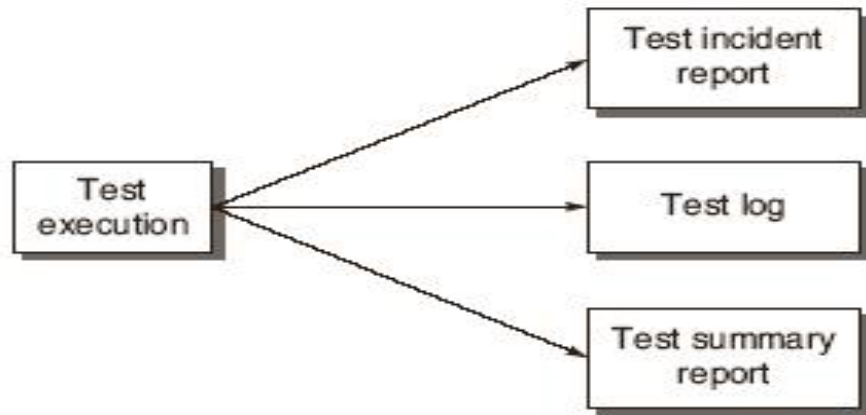


Fig: documents in Test Execution

Responsibilities at various levels for execution of the test cases are outlined in the following table.

Test Execution Level	Responsibility
Unit	Developer
Integration	Testers & Developers
System	Testers, developers, users
Acceptance	Testers, end-users

Fig(Tab): Testing level Vs. Responsibility

4. Test Review/Post-Execution:

This phase is to analyze bug related issues and obtain the feed back. As soon as the developer gets the bug-report.

performs the following activities:

Understanding the bug : The developer analyses the bug report and build an understanding of its.

Reproducing the bug: This is done to confirm the bug position and its where about so as to avoid the failures.

Analyze the nature and cause of a bug: After examining the failure of the bug, the developer starts debugging its symptoms and track back to the actual location of the error in the design.

After these ,the results from manual and automated testing can be collected and the following activities can be done:

Reliability Analysis (are the reliability goals being met or not), coverage analysis and overall defect analysis.

After fixing the bug , the developer reports to the testing team and the modified portion of the software is tested once again.

- **Reliability analysis :** It can be performed to establish whether the software meets the predefined reliability goals or not.

- **Coverage analysis :** It can be used as an alternative criterion to stop testing.

- **Overall defect analysis :** It can identify risk areas and help focus our efforts on quality improvement.

2.3.SOFTWARE TESTING METHODOLOGY

- Software testing methodology is the organization of software testing by means of which the test strategy and test tactics are achieved.
- Software testing methodology and a complete testing strategy is shown in the below figure

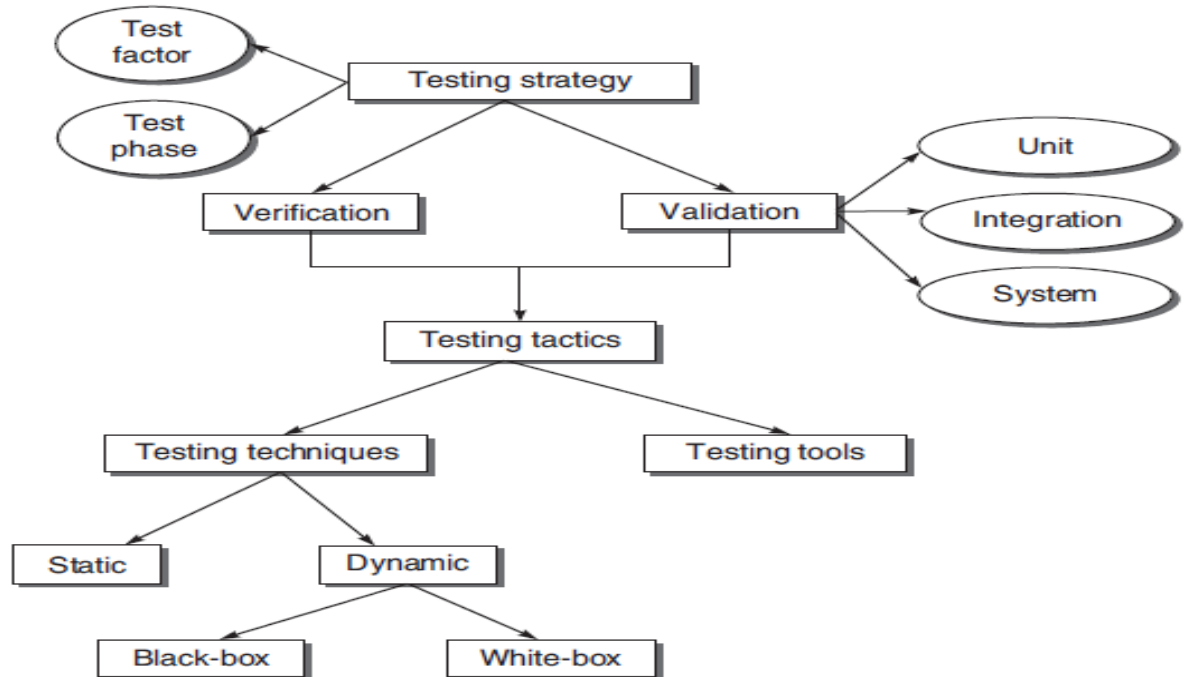


Fig: Testing Methodology

2.3.1. Software Testing Strategy

2.3.2. Test Strategy Matrix

2.3.3. Development of Test Strategy

2.3.4. Testing Life Cycle Model

2.3.5. Validation Activities

2.3.6. Testing tactics

2.3.7. Considerations in developing testing methodologies

2.3.1. Software Testing Strategy

- Testing strategy is the planning of the whole testing process into a well-planned series of steps.
- The components on which the testing strategy is based are discussed below:
 1. **Test factors:**
 - Test factors are risk factors or issues related to the system under development.
 - These factors are needed to be selected and ranked according to a specific system under development.
 - Testing process should reduce these test factors to a prescribed level.
 2. **Test Phase:**
 - It refers to the phases of SDLC where testing will be performed.

Testing strategy will be different for different models of SDLC. (Ex: Waterfall, spiral etc.)

2.3.2. Test Strategy Matrix:

A test strategy matrix identifies the concerns that will become the focus of test planning and execution. The matrix is prepared using test factors and test phase. The steps to prepare the test matrix are:

- select and rank test factors,
- Identify system development phases,
- Identify risks associated with the system under development.

Select and rank test factors

- Based on the test factors list, the most appropriate factors according to specific systems are selected and ranked from the most significant to the least.
- These are the rows of the matrix.

Identify system development phases

- Different phases according to the development model are listed as columns of the matrix.
- These are called test phases.

Identify risks associated with the system under development.

- In the horizontal column under each of the test phases, the test concern with the strategy used to address this concern is entered.
- The risks may include any events, actions, or circumstances that may prevent the test program from being implemented or executed according to a schedule.
- The purpose is to identify the concern that need to be addressed under a test phase.

Ex: Let us take a project as an example. Suppose a new operating system has to be designed, which needs attest strategy. The following is the example test strategy matrix

Test strategy matrix

Test Factors	Test Phase					
	Requirements	Design	Code	Unit test	Integration test	System test

Creating a Test Strategy: As shown in Fig given above,

a project of designing a new OS is taken as an example.

The steps to be used are:

1.Select and rank test factors: Portability is an important factor to be checked out for an OS. This factor is most important since an OS has to be compatible for different kinds of hardware configurations.

2.Identify the Test Phases: In this step, all test phases affected by the selected test factors are identified; these can be seen in **Fig given above**.

3.Identify the Risks: Risks are basic concerns associated with each factor in a phase and are expressed in the form of questions like: “Is testing performing successfully on INTEL,MIPS, and A MDH/W plat forms?”

4.Plan the test strategy for every risk identified: After identifying the risks, a plan strategy to tackle them is to be developed so that risks are mitigated.

Example test strategy matrix

Test Factors	Test Phase					
	Requirements	Design	Code	Unit test	Integration test	System test
Portability	Is portability feature mentioned in specifications according to different hardware?					Is system testing performed on MIPS and INTEL platforms?
Service Level	Is time frame for booting mentioned?	Is time frame incorporated in design of the module?				

2.3.3.Development of Test Strategy:

Test strategy should be such that testing starts at the very first phase of SDLC and continues till the end. Terms are:

Verification: The purpose here is to check the software at every development phase of SDLC in such a way that any defect can be detected at early stages and stopped. Verification can be stated as a set of activities that ensures correct implementation of functions in software.

Validation: It is used to test the software as a whole in accordance with the user’s expectations/specifications.

Barry Boehm says:

Verification means ‘are we building the product right?’ Validation means ‘are we building the right product?’

In another way,

Verification checks are we working in the right way and Validation ensures that the required goals have been achieved.

Verification is ‘Are we building the product right?’
Validation is ‘Are we building the right product?’

Verification	Validation
It check the product whether it meets the specific requirements of the particular phase	It check the final product whether it meets the business needs.
It checks whether the product is built as per the specified requirement and design specification.	It determines whether the software is fit for use and satisfies the business need.
Checks—Are we building the product right?	Checks—Are we building the right product!?
Involves all the static testing techniques	Includes all the dynamic testing techniques.
Examples includes reviews, inspection and walkthrough	Example includes all types of testing like smoke, regression, functional, system.

2.3.4. Testing Life Cycle Model

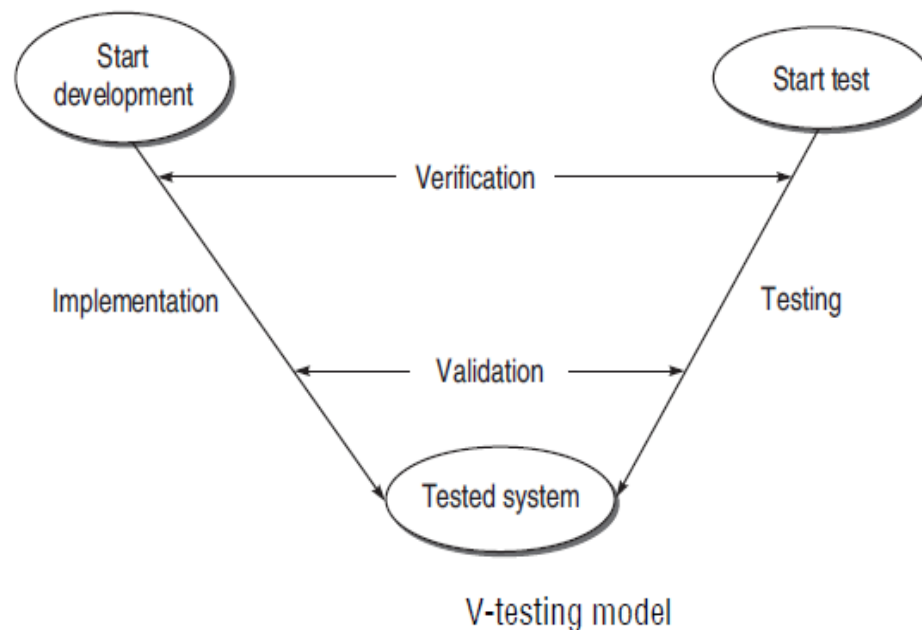
Verification and validation are the building blocks of a testing process. V& V Model is known as the Testing Life Cycle Model.

The formation of test strategy based on the two terms verification and validation. Life cycle involves continuous testing of the system during the development process (SDLC). At predetermined points, the results of the development process are inspected to determine the correctness of implementation. They also identify the errors as early as possible.

V-Testing Life Cycle Model

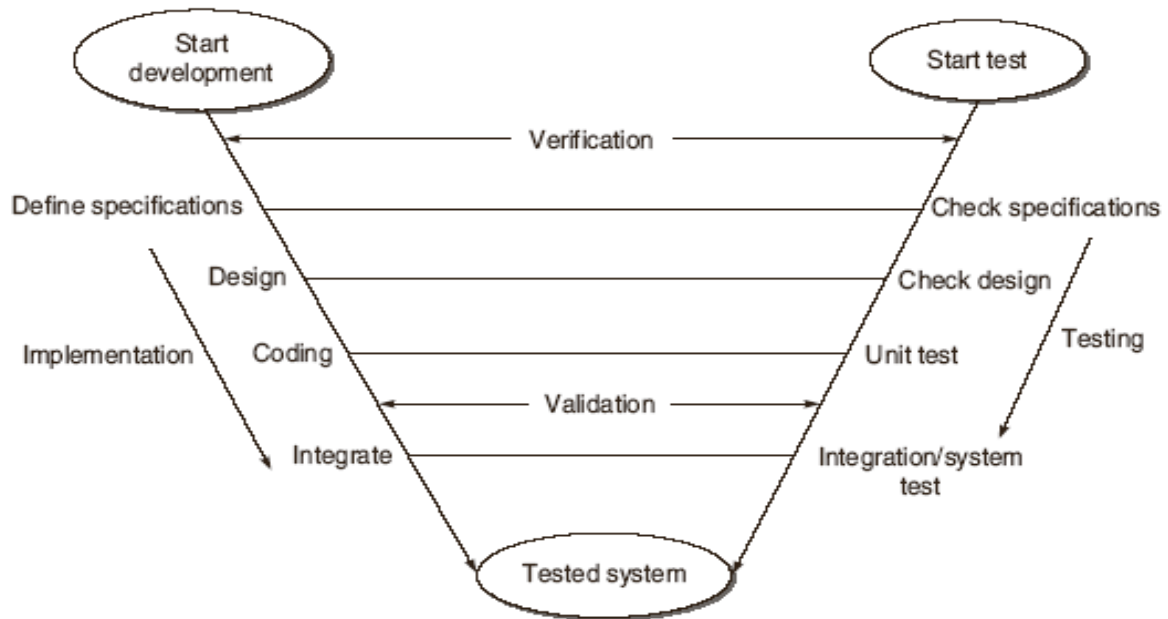
In V-Testing Model, developers attempt to implement the software, the testing team concurrently starts checking the software. Both teams stat at the same point using same information.

Fig: V-testing Model



It should be noted that the Figure given above can be expanded and elaborated further.

Fig: Expanded V-Testing Model:



The V&V process involves

(i) verification of every step of SDLC and (ii) validation of the verified system at the end.

2.3.5 Validation Activities:

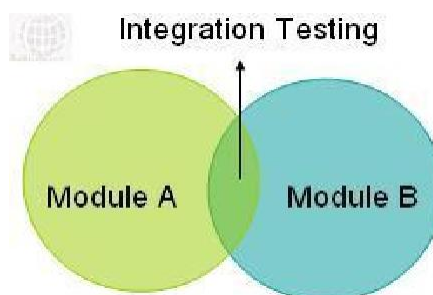
Validation has the following three activities which are known as the three levels of validation testing.

Unit Testing:

- Unit testing is a software development process in which the smallest testable parts of an application, called units, are individually and independently scrutinized for proper operation.
- Unit testing is often automated but it can also be done manually.

Integration testing:

- Integration testing tests integration or interfaces between components, interactions to different parts of the system such as an operating system, file system and hardware or interfaces between systems.
- After integrating two different components together we do the integration testing. As displayed in the image below when two different modules Module A and Module B are integrated then the integration testing is done.



System Testing (ST):

- System Testing is a black box testing technique performed to evaluate the complete system.
- In System testing, the functionalities of the system are tested from an end-to-end perspective.

2.3.6 Testing Tactics:

- The testing techniques can be categorized into two parts: (a) Static Testing, (b) Dynamic Testing.

Static Testing:

- Static testing is the testing of the software work products manually, but they are not executed.
- It starts early in the Life cycle and so it is done during the verification process.
- It does not need computer as the testing of program is done without executing the program.
- For example: reviewing, walk through, inspection, etc.

Dynamic Testing:

- Dynamic Testing is a kind of software testing technique using which the dynamic behavior of the code is analyzed.
- For Performing dynamic, testing the software should be compiled and executed and parameters such as memory usage, CPU usage, response time of the software are analyzed.
- Dynamic testing is further divided into : (i) Black-box testing (ii) White-box testing

Black-box testing:

- This technique takes care of the inputs given to a system and the output is received after processing in the system.
- It checks the functionality of the system only. That is why the term black-box is used. It is also known as *functional testing*. It is used for system testing under validation.

White-box testing:

- This technique complements black-box testing.
- Every design feature and its corresponding code is checked logically with every possible path execution. So, it takes care of the structural paths instead of just outputs.
- It is also known as *structural testing* and is used for unit testing under verification.

Testing Tools:

Testing tools provide the option to automate the selected testing technique with the help of tools. The tester should first understand the testing techniques and then go for the tools that can be used with each of the techniques.

2.3.7. Considerations in developing testing methodologies:

The considerations in developing a testing methodology are

Determine Project risks:

- A test strategy is developed with the help of another team familiar with the business risks associated with the software.
- The major issue in determining the strategy is to identify the risks associated with the project. What are the high priority risks? What are the consequences, if risks are not handled?

Determine the type of development project:

The environment or methodology to develop software also affects the testing risks.

Note that risks associated with a new project will be different from that of purchased software.

Identify test activities according to SDLC Phase:

- After identifying all the risk factors in an SDLC phase, testing can be started in that phase.
- All the testing activities must be recognized at all the SDLC phases.

Build the Test plan:

A tactical test plan is required to start testing.

This test plan provides:

- Environment and pre-test background
- Overall objectives
- Test team
- Schedule and budget showing detailed dates for the testing
- Resource requirements including equipment, software, and personnel
- Testing materials including system documentation, software to be tested, test inputs, test documentation, test tools
- Specified business functional requirements and structural functions to be tested
- Type of testing technique to be adopted in a particular SDLC phase or what are the specific tests to be conducted in a particular phase

III Verification and Validation

Verification and Validation:

1. Verification & Validation Activities.
2. Verification.
3. Verification of Requirements.
4. High level and low level designs.
5. verifying code.
6. Validation.

A V-diagram provides the following insights about software testing:

Testing can implemented in the same flow as for the SDLC

Testing can be broadly planned in two activities, namely verification and validation.

Testing must be performed at every step of SDLC.

V-diagram supports the concept of early testing.

V-diagram supports parallelism in the activities of developers and testers.

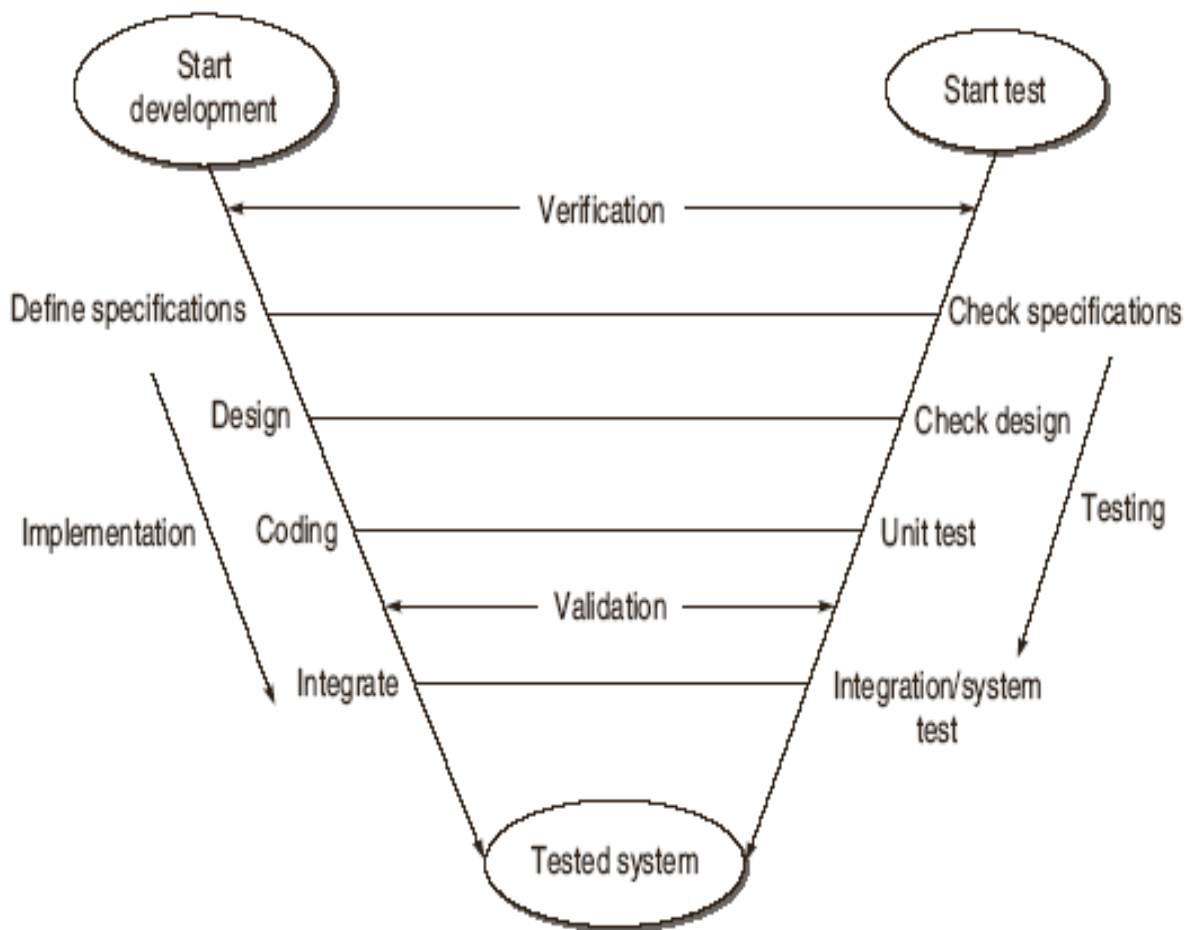


Fig: V-testing

3.1.Verification& Validation Activities (V&V):

V&V activities can be understood in the form of a diagram shown below to understand V&V activities first need to understand SDLC phases. The following are SDLC phases;

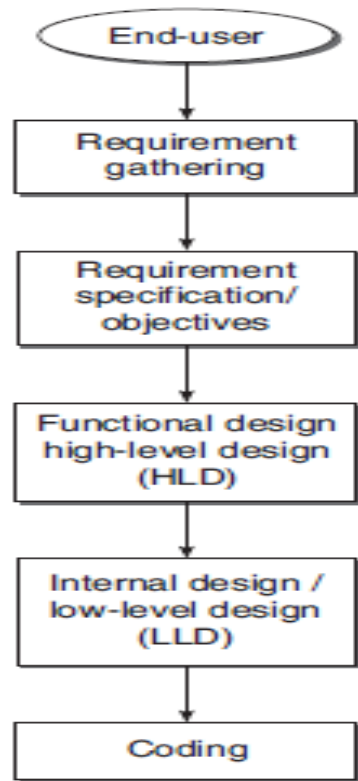


Fig :SDLC phases

Requirements Gathering:

The needs of user are gathered and translated into written set of requirements.

This requirements are prepared from the user's viewpoint only and do not include any technicalities according to the developer.

Requirement Speciation or Objectives:

1. In this phase, all the user requirements are specified in developer's terminology.
2. The specified objectives which are going to be developed are prepared in the form of a document known as **software requirement specification (SRS)**.

Functional design or high-level design

1. Functional design is the process of translating user requirements into a set of external interfaces.
2. The output of the process is the functional design which describes the product's behavior as seen by an observer external to the product.
3. The *high-level design* is prepared with SRS into a usable product.

4. In HLD, the software system architecture is prepared and broken into independent modules. Thus, an HLD document will contain the following items at a macro level:
 1. Overall architecture diagrams along with technology details.
 2. Functionalities of the overall system with the set of external interfaces.
 3. List of modules.
 4. Brief functionality of each module.
 5. Interface relationship among modules including dependencies between modules, database tables identified along with key elements.

Internal design or low-level design

- HLD provides the macro-level details of a system, an HLD document cannot be given to programmers for coding.
- So the analysts prepare a micro-level design document called *internal design* or *low-level design* (LLD).
- This document describes each and every module in an elaborate manner, so that the programmer can directly code the program based on this. There may be at least one separate document for each module.

Coding

- If an LLD document is prepared for every module, then it is easy to code the module.
- Thus in this phase, using design document for a module, its coding is done.

After understanding all the SDLC phases, we need to put together all the verification activities. Verification activities are performed almost at every phase; therefore all the phases of SDLC will be verified as shown below.

Along with these verification activities performed at every step, the tester needs to prepare some test plans which will be used in validation activities performed after coding the system. These test plans are prepared at every SDLC phase.

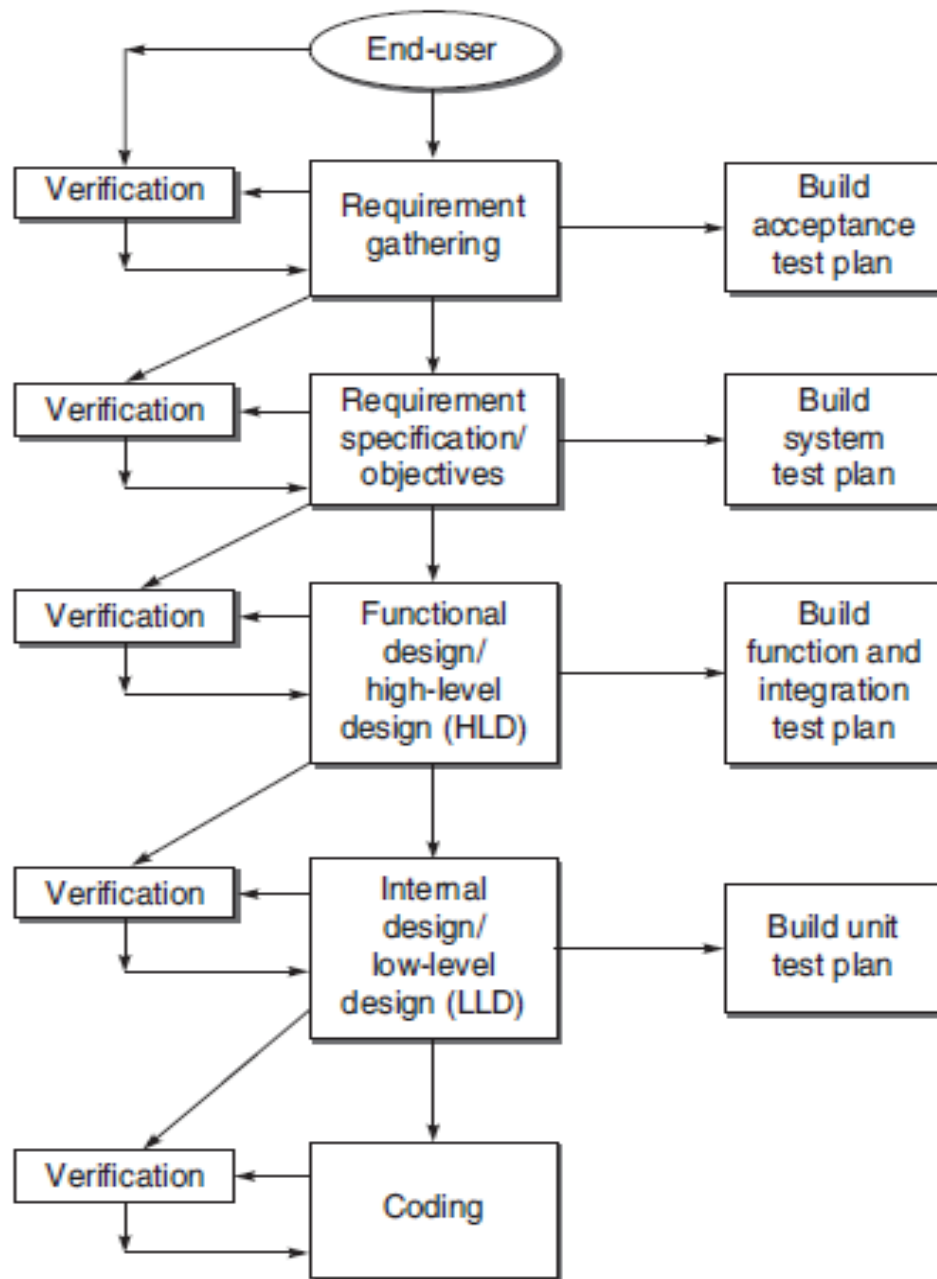


Fig : Tester performs verification and prepares test plan during every SDLC phase

When the coding is over for a unit or a system, and parallel verification activities have been performed, then the system can be validated. These are executed with the help of test plans prepared by the testers at every phase of SDLC.

The validation activities are executed with the help of test plans. This brings up the complete V& V activities diagram.

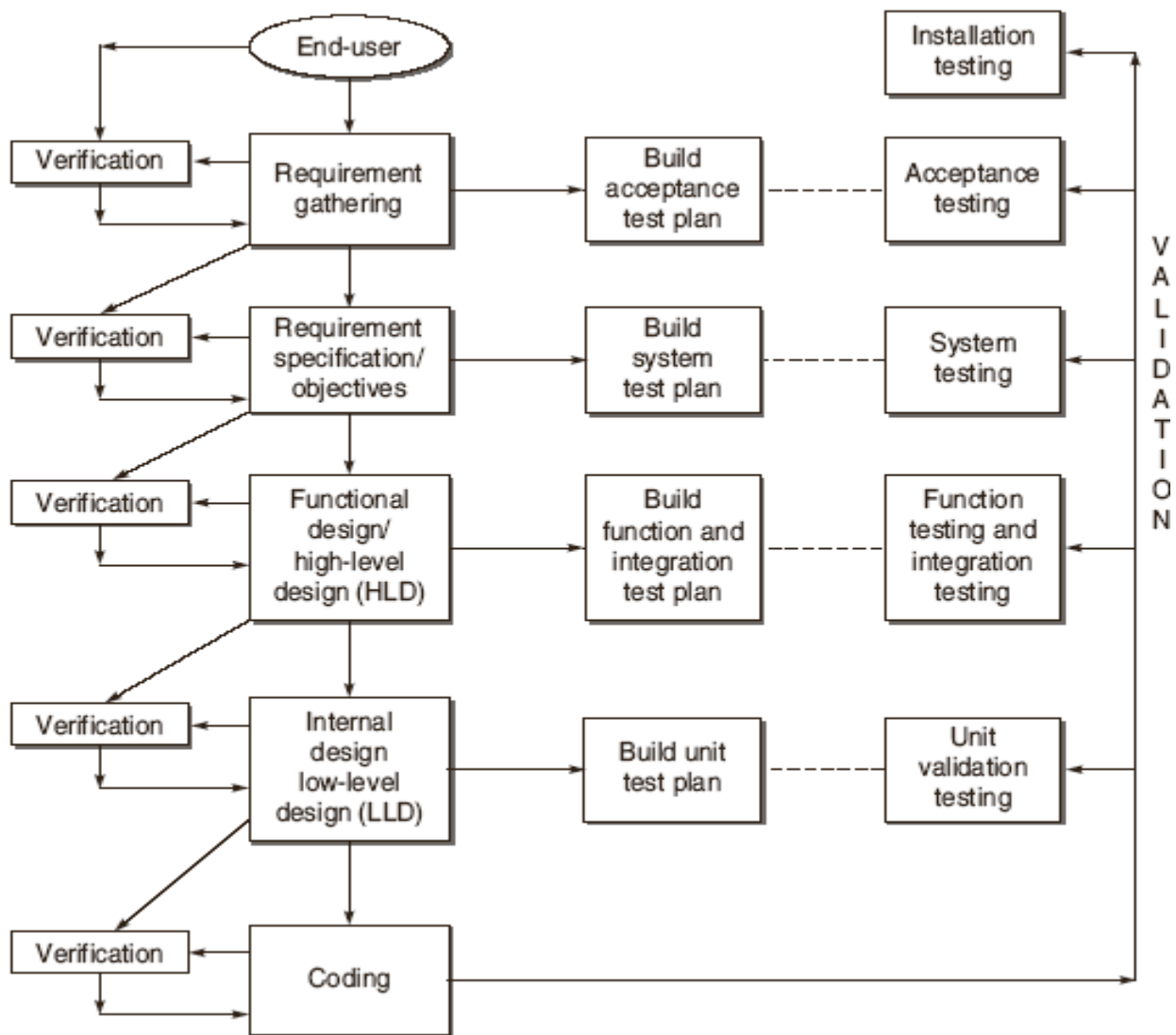


Fig: V&V diagram

3.2 Verification

- Verification is a set of activities that ensures correct implementation of specific functions in software. It is mandatory that verification is performed at every step of SDLC.
- Verification is needed for the following:
 1. If verification is not performed at early stages, there are always a chance of mismatch between the required product and the delivered product. Suppose, if requirements are not verified, then it leads to situations where requirements and commitments are not clear. These things become important in case of non-functional requirements and they increase the probability of bugs.
 2. Verification exposes more errors.
 3. Early verification decreases the cost of fixing bugs.

4. Early verification enhances the quality of software.
- Verification goals are the following:
 1. **Everything Must be Verified:** All the SDLC phases and all the products of these processes must be verified.
 2. **Results of Verification May Not be Binary:**
 - Verification may not just be the acceptance or rejection of a product.
 - There are many verification activities whose results cannot be reduced to a binary answer.
 - For example, sometimes correctness of a requirement cannot be rejected or accepted but can be accepted with a degree of satisfaction or rejected with a certain degree of modification.
 3. **Even Implicit Qualities Must be Verified:**
 - The qualities desired in the software are explicitly stated in the SRS.
 - But those requirements which are implicit and not mentioned anywhere must also be verified.

3.3 Verification of Requirements

- Verification of Requirements means, all the requirements gathered from the user's viewpoint is verified. For this purpose, an acceptance criterion is prepared.
- An acceptance criterion defines the goals and requirements of the proposed system and acceptance limits for each of the goals and requirements.
- The acceptance criteria mostly used in case of real-time systems where performance is a critical issue in certain events.
- For example, if a real-time system has to take decision for a weapon within x seconds, then any performance below this would lead to rejection of the system.
- Therefore, it can be concluded that acceptance criteria for a requirement must be defined by the designers of the system.
- The tester works in parallel by performing the following two tasks:
 1. The tester reviews the acceptance criteria in terms of its completeness, clarity, and testability.
 2. The tester prepares the **Acceptance Test Plan** which is referred at the time of Acceptance Testing.

3.3.1 Verification of Objectives

- After gathering requirements, specific objectives are prepared considering every specification. These objectives are prepared in a document called software requirement specification (SRS).
- In this activity also, two parallel activities are performed by the tester:

1. The tester verifies all the objectives mentioned in SRS. The purpose of this verification is to ensure that the user's needs are properly understood before proceeding with the project.
2. The tester also prepares the *System Test Plan* which is based on SRS. This plan will be referenced at the time of *System Testing*.

3.3.2.How to Verify Requirements and Objectives?

- Requirement and objectives verification has a high ability of detecting bugs. Therefore, requirements must be verified.
- The testers use the SRS for verification of objectives
- An SRS can be verified, if and only if, every requirement stated herein can be verified. A requirement can be verified, if, and only if, there is some procedure to check that the software meets its requirement.
- The points against which verification of requirement will be done? Following are the points against which every requirement in SRS should be verified:

Correctness:

- There are no tools or procedures to measure the correctness of a specification. The tester uses his or her intelligence to verify the correctness of requirements.
- Following are some points to verify the correctness
 1. Testers should refer to other documentations or appropriate standards and compare the specified requirement with them.
 2. Testers can interact with customers or users, if requirements are not well-understood.
 3. Testers should check the correctness in the sense of realistic requirement. If the tester feels that a requirement cannot be understood clearly using existing hardware and software technology, it means that it is unrealistic. In that case, the requirement should either be updated or removed from SRS.

Unambiguous:

- A requirement should be verified such that it does not provide too many meanings. It should not create redundancy in specifications.
- Each characteristic should be described using a single term, otherwise ambiguity or redundancy may cause bugs in the design phase. The following must be verified:
 1. Every requirement has only one interpretation.
 2. Each characteristic of the final product is described using a single unique term.

Consistent:

- No specification should contradict or conflict with another. Conflicts produce bugs in the next stages, therefore they must be checked for the following:

1. Real-world objects conflict, for example, one specification recommends mouse for input, another recommends joystick.
2. Logical conflict between two specified actions, e.g. one specification requires the function to perform square root, while another specification requires the same function to perform square operation.
3. Conflicts in terminology should also be verified. For example, at one place, the term *process* is used while at another place, it has been termed as *task* or *module*.

Updation:

- Requirement specifications are not stable, they may be modified or another requirement may be added later. Therefore, if any updation is there in the SRS, then the updated specifications must be verified.
 1. If the specification is a new one, then all the above mentioned steps and their feasibility should be verified.
 2. If the specification is a change in an already mentioned specification, then we must verify that this change can be implemented in the current design.

Traceability:

- The traceability of requirements must also be verified such that the origin of each requirement is clear and also whether it makes easy referencing in future development or increase documentation.
- The following two types of traceability must be verified:

Backward traceability Check that each requirement references its source in previous documents.

Forward traceability Check that each requirement has a unique name or reference number in all the documents.

3.4 Verification of High-Level Design

- All the requirements mentioned in the SRS document are addressed in this phase and work in the direction of designing the solution.
- The architecture and design is documented in another document called the *software design document* (SDD).
- the verification of requirements, the tester is responsible for two parallel activities in this phase as well:
 1. The tester verifies the high-level design. Since the system has been decomposed in a number of sub-systems or components, the tester should verify the functionality of these components. the system is considered with no low-level details. So all the interfaces and interactions of user/customer are specified in this phase. The tester verifies that all the components and their interfaces are in tune with requirements of the user.

2. The tester also prepares a *Function Test Plan* which is based on the SRS.

3.4.1 How to Verify High-Level Design?

- High-level design takes the second place in SDLC, wherein there is a high probability of finding bugs.
- High-level design must be verified as a next step in early testing. The design is specified in a formal way, design cannot be verified. So SDD is referred for design verification.
- If a bug goes undetected in the high-level design phase, then its cost of fixing increases with every phase. Therefore, verification for high-level design must be done very carefully. This design is divided in three parts

Data Design

- It creates a model of data and/ information that is represented at a high level of abstraction (the customer/user's view of data).

Architectural Design

- It focuses on the representation of the structure of software components, their properties, and interactions.

Interface Design

- It creates an effective communication medium between the interfaces of different software modules, interfaces between the software system and any other external entity, and interfaces between a user and the software system.

Verification of high-level design will consider all the three designs and this will become the basis for effective verification.

Verification of Data Design

The points considered for verification of data design are as follows:

- Check whether the sizes of data structure have been estimated appropriately.
- Check the consistency of data formats with the requirements.
- Check whether data usage is consistent with its declaration.
- Check the relationships among data objects in data dictionary.
- Check the consistency of databases and data warehouses with the requirements specified in SRS.

Verification of Architectural Design

The points considered for the verification of architectural design are:

- Check that every functional requirement in the SRS has been take care of in this design.
- Check whether all exceptions handling conditions have been taken care of.
- Architectural design deals with the classification of a system into sub-systems or modules check the functionality of each module according to the requirements specified.

Verification of User-Interface Design

The points to be considered for the verification of user-interface design are:

- Check all the interfaces between modules according to the architecture design.
- Check all the interfaces between human and computer.
- Check all the above interfaces for their consistency.
- Check the response time for all the interfaces are within required ranges. It is very essential for the projects related to real-time systems where response time is very crucial.
- For a Help Facility, verify the following:
 - i. The representation of Help in its desired manner.
 - ii. The user returns to the normal interaction from Help.
- For error messages and warnings, verify the following:
 - i. Whether the message clarifies the problem.
 - ii. Whether the message provides constructive advice for recovering from the error.
- For typed command interaction, check the mapping between every menu option and their corresponding commands.

3.5 Verification of Low-Level Design

- In LLD, a detailed design of modules and data are prepared such that operational software is ready.
- For this, SDD is preferred where all the modules and their interfaces are defined.
- Every operational detail of each module is prepared. The details of each module or unit is prepared in their separate SRS and SDD.

Testers also perform the following parallel activities in this phase:

- The tester verifies the LLD. The details and logic of each module is verified such that the high-level and low-level abstractions are consistent.
- The tester also prepares the *Unit Test Plan* which will be referred at the time of *Unit Testing*

3.5.1 How to Verify Low-Level Design?

- This is the last pre-coding phase where internal details of each design entity are described.
- For verification, the SRS and SDD of individual modules are referred.
- Some points to be considered are listed below:
 1. Verify the SRS of each module.
 2. Verify the SDD of each module.
 3. In LLD, data structures, interfaces, and algorithms are represented by design notations; verify the consistency of every item with their design notations.

3.6 How to Verify Code?

- Coding is the process of converting LLD specifications into a specific language.
- This is the last phase when we get the operational software with the source code.
- Therefore, the code must also be verified. The points against which the code must be verified are:
 - i. Check that every design specification in HLD and LLD has been coded using traceability matrix.
 - ii. Examine the code against a language specification checklist.
 - iii. Code verification can be done most efficiently by the developer, as he has prepared the code. Some points against which the code can be verified are:
 - a. Misunderstood or incorrect arithmetic precedence.
 - b. Mixed mode operations.
 - c. Incorrect initialization.
 - d. Incorrect symbolic representation of an expression.
 - e. Different data types.
 - f. Improper or non-existent loop termination
 - g. Failure to exit.

Two kinds of techniques are used to verify the coding: (a) static testing, and (b) dynamic testing.

Static testing techniques

This technique does not involve actual execution. It considers only static analysis of the code or some form of conceptual execution of the code.

Dynamic testing techniques

It executes the code on some test data. The developer is the key person in this process who can verify the code of his module by using the dynamic testing technique.

3.6.1 Unit Verification

- Verification of coding cannot be done for the whole system. The system is divided into modules.
- Therefore, verification of coding means the verification of code of modules by their developers. This is also known as unit verification testing.
- Listed below are the points to be considered while performing unit verification
 1. Interfaces are verified to ensure that information properly flows in and out of the program unit under test.
 2. The local data structure is verified to maintain data integrity.
 3. Boundary conditions are checked to verify that the module is working fine on boundaries also.
 4. All independent paths through the control structure are exercised to ensure that all statements in a module have been executed at least once.

5. All error handling paths are tested.

Unit verification is largely white-box oriented.

3.7 Validation

- Validation is a set of activities that ensures the software has been built right and is traceable to customer requirements.
- Validation testing is performed after the coding is over. The reasons are:
 1. To determine whether the product satisfies the users' requirements, as stated in the requirement specification.
 2. To determine whether the product's actual behavior matches the desired behavior.
 3. It is not always certain that all the stages till coding are bug-free. The bugs those are still present in the software after the coding phase need to be uncovered.
 4. Validation testing provides the last chance to discover bugs, otherwise these bugs will move to the final product released to the customer.
 5. Validation enhances the quality of software.

3.7.1 Validation Activities

- The validation activities are divided into Validation Test Plan and Validation Test Execution

Validation Test Plan

- It starts as soon as the first output of SDLC, i.e. the SRS, is prepared.
- In every phase, the tester performs two parallel activities—verification at that phase and the corresponding validation test plan.
- For preparing a validation test plan, testers must follow the points:
 1. Testers must understand the current SDLC phase.
 2. Testers must study the relevant documents in the corresponding SDLC phase.
 3. On the basis of the SDLC phase and related documents, testers must prepare the related test plans which are used at the time of validation testing.

The following test plans have been recognized which the testers have already prepared with the SDLC phases:

Acceptance test plan

- This plan is prepared in the requirement phase according to the acceptance criteria prepared from the user feedback.
- This plan is used at the time of Acceptance Testing.

System test plan

- This plan is prepared to verify the objectives specified in the SRS.
- The plan is used at the time of System Testing.

Function test plan

- This plan is prepared in the HLD phase.
- In this plan, test cases are designed and every type of functionality can be tested.
- The plan is used at the time of Function Testing.

Integration test plan

- This plan is prepared to validate the integration of all the modules such that all their interdependencies are checked.
- It also validates whether the integration is in conformance to the whole system design.
- This plan is used at the time of Integration Testing.

Unit test plan

- This plan is prepared in the LLD phase.
- It consists of a test plan of every module in the system separately.
- Unit test plan of every unit or module is designed such that every functionality related to individual unit can be tested.
- This plan is used at the time of Unit Testing.

Validation Test Execution

- Validation test execution can be divided in the following testing activities:

Unit validation testing

- The testing strategy is to first focus on the smaller building blocks of the full system.
- A unit or module must be validated before integrating it with other modules.
- Unit validation is the first validation activity after the coding of one module is over.
- The motivation for unit validation as compared to the whole system are as follows:
 1. If the whole software is tested at once, then it is very difficult to trace the bug. Thus, debugging becomes easy with unit testing.
 2. In large-scale projects, a number of modules may be there and each module may be composed of tens of thousands of lines of code. In such a case, it is not possible for a single developer to develop all the modules. There is a team of developers working on separate modules. This requires parallelism in software development. Thus, every module can be developed and tested independently.

Integration testing

- It is the process of combining and testing multiple components or modules together.
- The individual tested modules, when combined with other modules, are not tested for their interfaces.

- Therefore, they may contain bugs in integrated environment. Thus, to uncover the bugs that are present when unit tested modules are integrated.

Function testing

- The objective of function test is to measure the quality of the functional (business) components of the system.
- Tests verify that the system behaves correctly from the user/business perspective and functions according to the requirements.
- The function test must determine if each component or business event:
 1. Performs in accordance to the specifications.
 2. Responds correctly to all the conditions that may be presented by incoming events/data.
 3. Moves the data correctly from one business event to the next.
 4. Is initiated in the order required to meet the business objectives of the system.

System testing

- System testing is actually tests whose primary purpose is to fully work out a computer-based system.
- System testing intention is to test the whole system on various grounds where bugs may occur.
- For example, if the software fails in some conditions, how does it recover?
- Another example is protection from improper penetration, i.e. how secure the whole system is.

Acceptance testing

- In the requirement phase of SDLC, acceptance criteria for the system to be developed are mentioned in one agreement by the customer.
- It can be tested against the acceptance criteria contracted with the customer. This is called acceptance testing.
- Acceptance testing can be defined as the process of comparing the final system with the needs of customer as agreed on the acceptance criteria.

Installation testing

- The installation testing does not test the system, but it tests the process of making the software system operational.
- The installation process must identify the steps required to install the system.
- For example, it may be required that some files may have to be converted into another format. Or it may be possible that some operating software may be required.